

Agent 如何做好长程任务

一次 context engineering 的实战汇报



一条长程轨迹，被切成短段——段间落盘，段间校验

项目背景

「贯穿案例」 **video2pdf**: 用 agent 工作流把 B 站 / YouTube 视频自动转成图文并茂的中文 PDF 讲义，本场案例全部来自这个真实项目。

「项目起点」 一个开源 skill —— github.com/wdkns/wdkns-skills 的 `bilibili-render-pdf`。

「原始流程」 探测元数据 → 下载字幕 (CC → Whisper → 纯视觉三级兜底) 与视频封面 → 长视频切段、多 subagent 并行分析

→ 按字幕时间戳抽关键帧配图 → LaTeX 中文讲义写作 → 检查清单后交付 PDF

workspace/Claude for Long_Horizon Tasks _ Lance Martin_ Anthropic_20260728_161214/review/acceptance/attempts/

```
$ ls attempts/
attempt_01 attempt_02

attempts/
├─ attempt_01/
│  ├─ acceptance_report.json      23 KB    # "overall_status":"fail" ← 第一次验收
│  ├─ changed_files.json
│  └─ repair_brief.md             16 KB    # 修复任务书（三条 failed_criteria）
└─ attempt_02/
   ├─ acceptance_report.json      26 KB    # "overall_status":"fail" ← 第二次验收
   ├─ changed_files.json
   └─ repair_brief.md             11 KB

# 第三次验收才通过：通行证据写回上级 review/acceptance/acceptance_report.json ("overall_status": "pass"),
# 两次失败的完整档案保留在 attempts/ 内，永不删除。
```

FAIL ① 元写作混入正文（P21）+ 两图标题越界（图9/图20）

FAIL ② 术语首用重复两遍：「无状态运行框架 无状态运行框架」（P7 §2.2）

一个讲「长程任务」的视频，自己的验收挂了一次——失败尝试全部留档

attempts/attempt_01/repair_brief.md · 第 1-13 行

```
# Repair Brief attempt_01
Repair subagents may inspect and modify only files
inside this video output directory.
```json
{
 "attempt" : "attempt_01" ,
 "failed_criteria" : [
 "no_meta_writing_content" ,
 "figure_visual_integrity" ,
 "credibility_disclosure_placement"
],

第一次验收失败：三条判据被机器留档，
 brief 直接交给 repair subagent 执行。
```

review/acceptance/repair\_handoff\_attempt\_01.md · 第 1-9 行

```
Acceptance Repair Handoff – attempt 01
Result

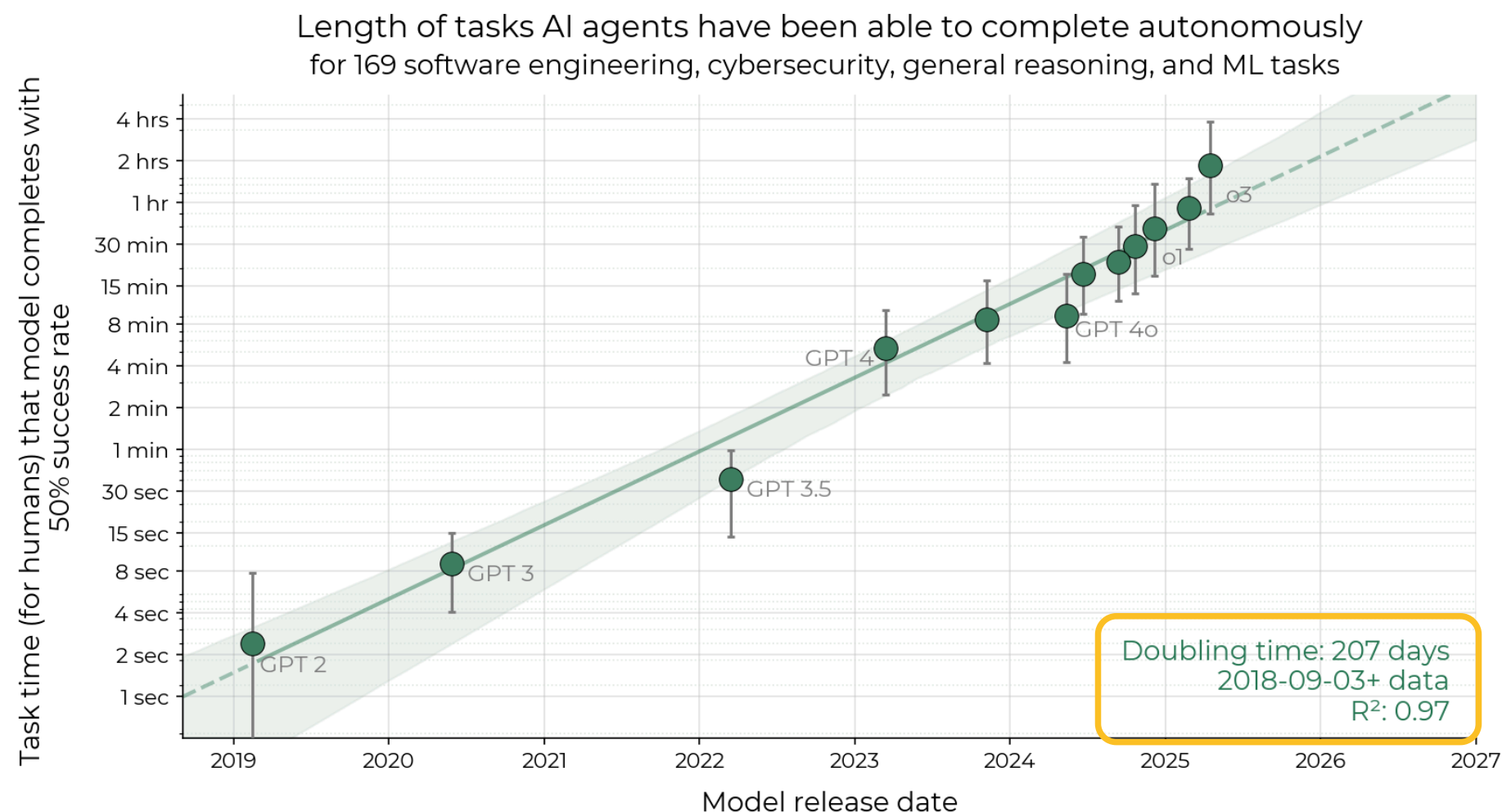
The three defects recorded by Acceptance attempt 01
are repaired. The final source package is ready for a
fresh guarded recompilation. The existing 27-page PDF
and its rendered acceptance pages predate these repairs
and remain stale.

修复完成也留档：改了什么、什么已过期（stale），
 交接给下一次「带守卫的重编译 + 重新验收」。
```

「失败 → 修复」闭环，每次 FAIL 都一样：  
① repair brief 写死：哪条规则挂了、证据在哪、允许怎么修  
② 修复 subagent 只准改产物，改完重走一次完整受控验收  
③ 两修两验，第三次通过，放行交付（attempt 上限 3）

失败留档 (repair\_brief) + 修复留档 (repair\_handoff) ——可靠性是被门禁逼出来的，不是口号

# 「长」如何度量——用「人类完成同一任务所需的时间」给任务长度标刻度



**定义** | 任务长度 = 人类专家完成这个任务需要多长时间

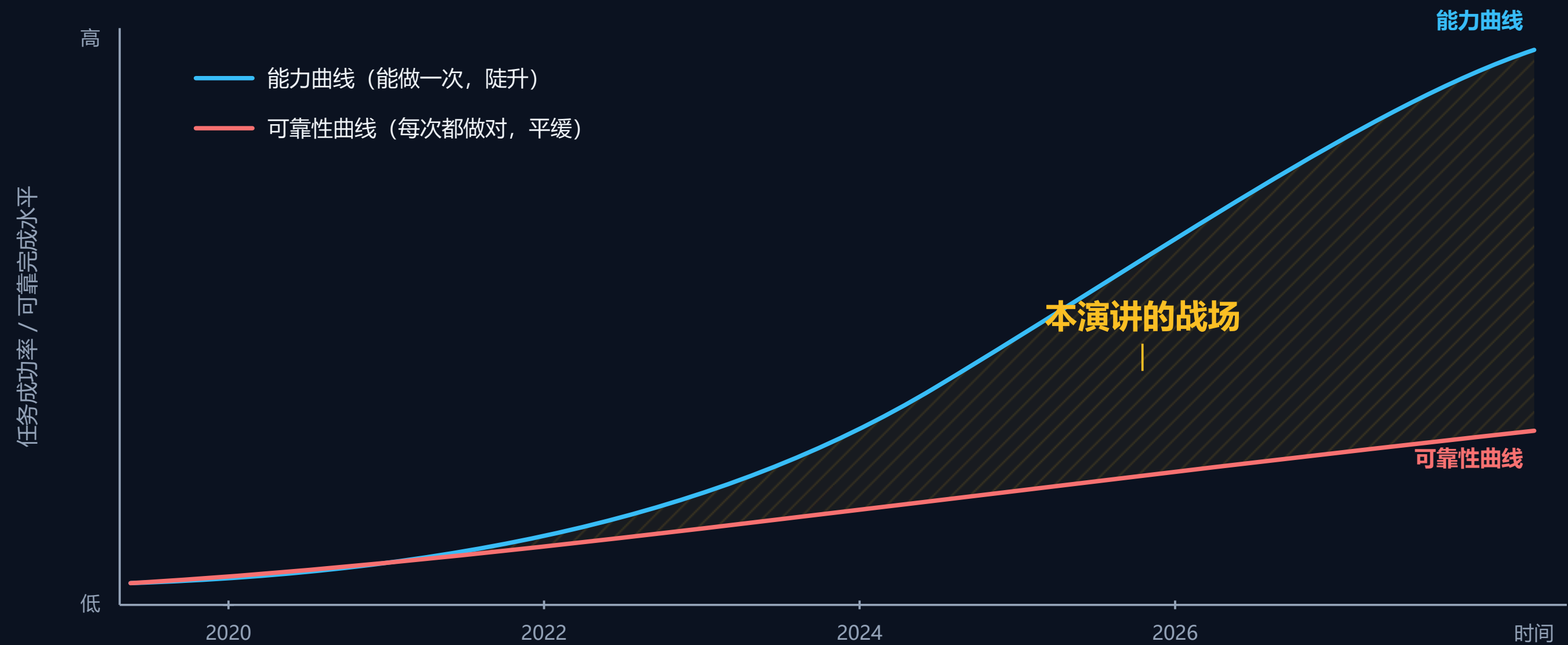
**读点** | 图中每个点 = 一个模型在对应长度任务上的成功率

**读法** | 2025 年初的前沿模型，在人类约 50 分钟的任务上，成功率只有 50%

**趋势** | 这个「50% 成功率对应的任务时长」约每 7 个月翻一倍 (doubling time  $\approx$  207 天)

Kwa et al. (METR), Measuring AI Ability to Complete Long Tasks, arXiv:2503.14499 (2025) — Figure 1 原图

# 瓶颈不是能力，是可靠性



切分保聪明

让每个 agent 只干小任务，保住它的聪明区间

校验保正确

让判断的人和干活的人分开，保住正确

落盘保连续

让每一步状态都写在磁盘上，保住连续性

# 今天的地图

全程约 60 分钟 · 每段都有 video2pdf 实战作证



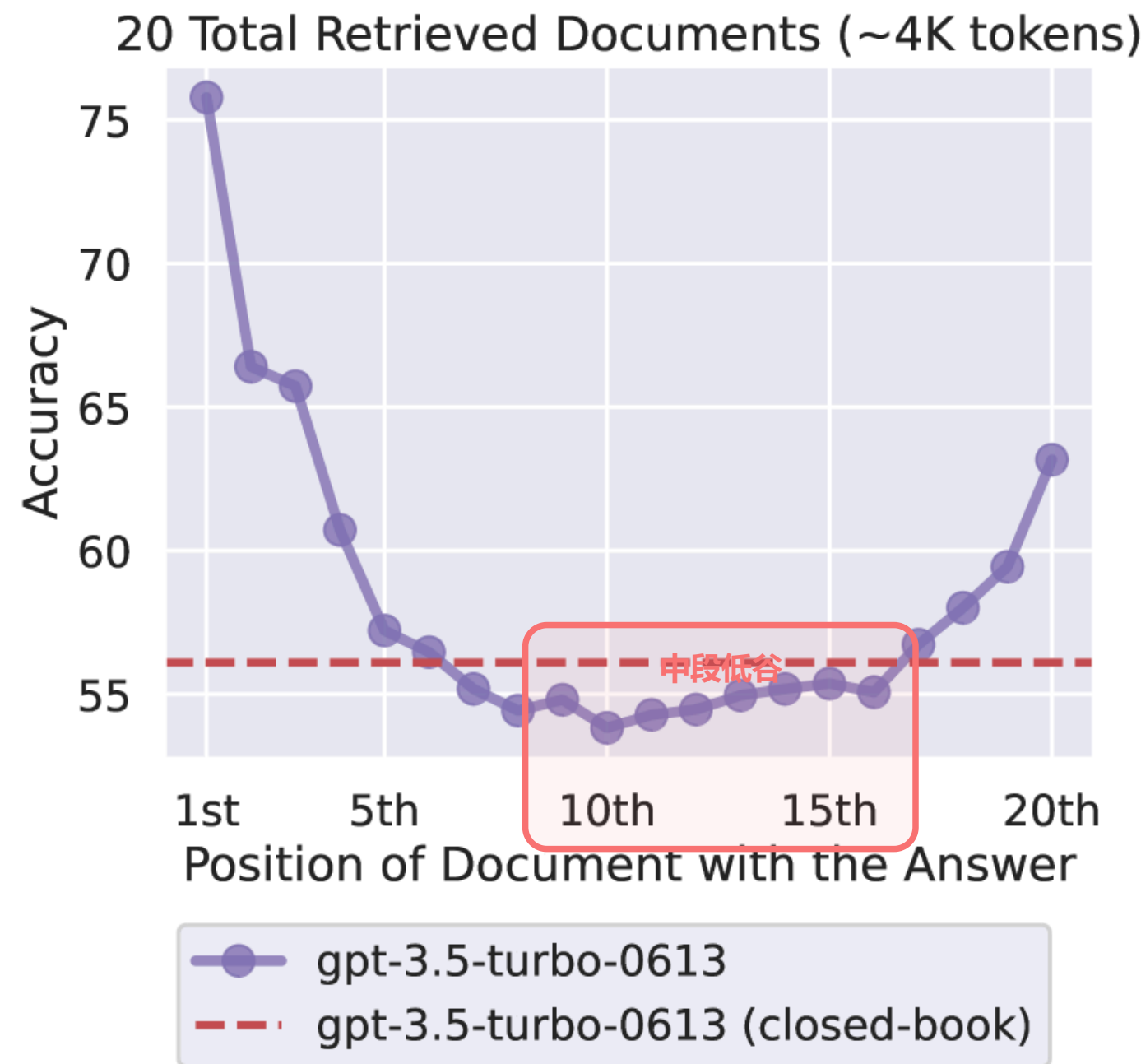
第 1 部分 · Agent 失败模式 · 10 分钟

# 为什么 agent 跑不长

问题不在模型不够大，在上下文本身



# 信息位置决定召回率：关键信息应该放在哪里？



## 论文事实: Lost in the Middle

长上下文中，信息的位置直接影响召回：关键信息落在中段时，检索与问答准确率显著低于首尾，呈 **U 形曲线**。而长程任务的关键决策信息，往往恰好落在中段。

来源：Liu et al., Lost in the Middle, arXiv:2307.03172, TACL 2024 (原图见左)

## 为什么这对 agent 致命

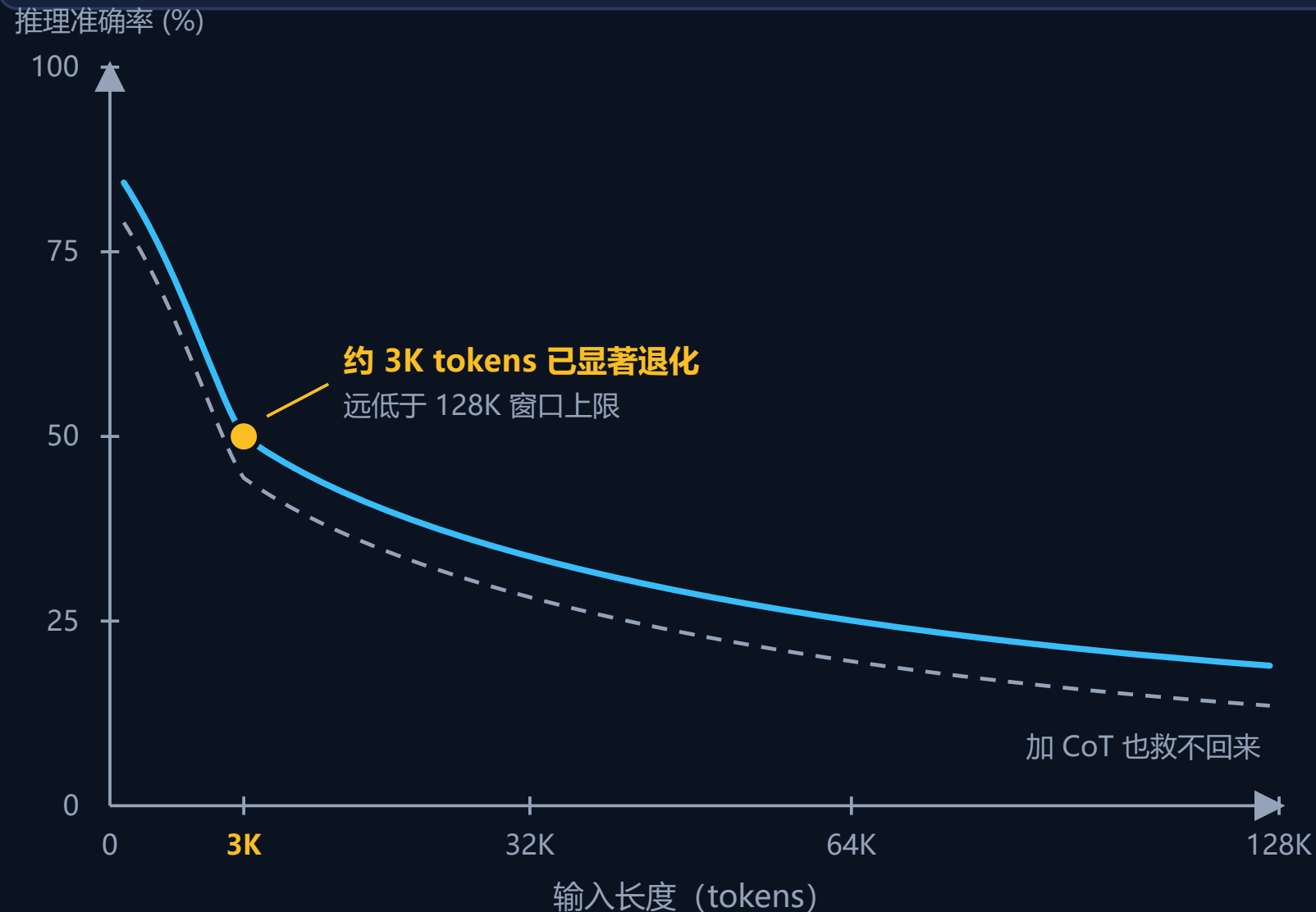
agent 的上下文长什么样？开头是系统提示与任务说明，结尾是最近几步对话；中间是几小时前下载的日志、几十步之前的工具输出、某步定下的关键约定——恰恰是「中途做下的关键决策所依赖的信息」，全部落在模型最不可靠的位置。

上下文越长，中段越大，关键信息埋得越多，失焦越严重；模型的注意力天然偏向**开头与结尾**——首尾的信息更容易被它「看见」。

# 最长上下文 ≠ 有效上下文：模型存在「傻区」

**实验设计：**任务一个字都不改、难度一点不加——只做一件事：往输入里填充无关文本，把它人为拉长，再看推理准确率怎么变。

**结果：**约 3K tokens 已经显著退化；连 chain-of-thought（思维链）都救不回来。



## 社区观点 · 经验法则，非论文结论

窗口标称已到 **1M token**，但社区经验法则认为「聪明区」大约只有 **150K**。

Matt Pocock 的「dumb zone / 笨区」——即本页「傻区」：半年前他的建议还是 100K——数字在漂移，因模型与任务而异。**当锚点，别当定律。**

越过边界：**幻觉增多、检索变弱。**

出现「忠实性幻觉」（输出与所给材料不一致），就是已经掉进傻区的信号。

—— 以上为社区经验法则，供参考；  
论文实验证据见左图曲线。

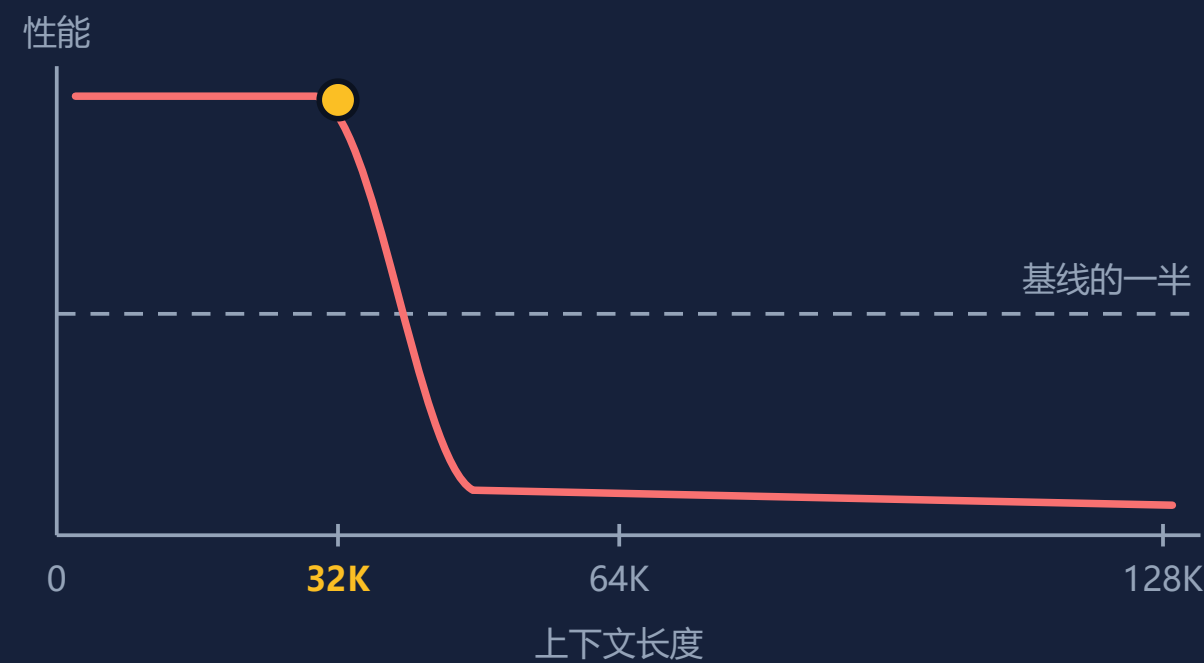
来源：Levy, Jacoby & Goldberg, Same Task, More Tokens, arXiv:2402.14848 (ACL 2024); 社区观点：Matt Pocock 「dumb zone」 (经验法则，非论文结论)

# 病灶三：任务越需要真推理，退化来得越早（NoLiMa / Context Rot）

## NoLiMa：32K 处的悬崖

实验：任务的字面匹配极容易，但答案需要跨长上下文做一步推理（问题与答案刻意去掉字面重叠，逼模型真推理）。

读图：短上下文的高性能平台在 32K 处陡降成悬崖——任务越需要真推理，退化来得越早。

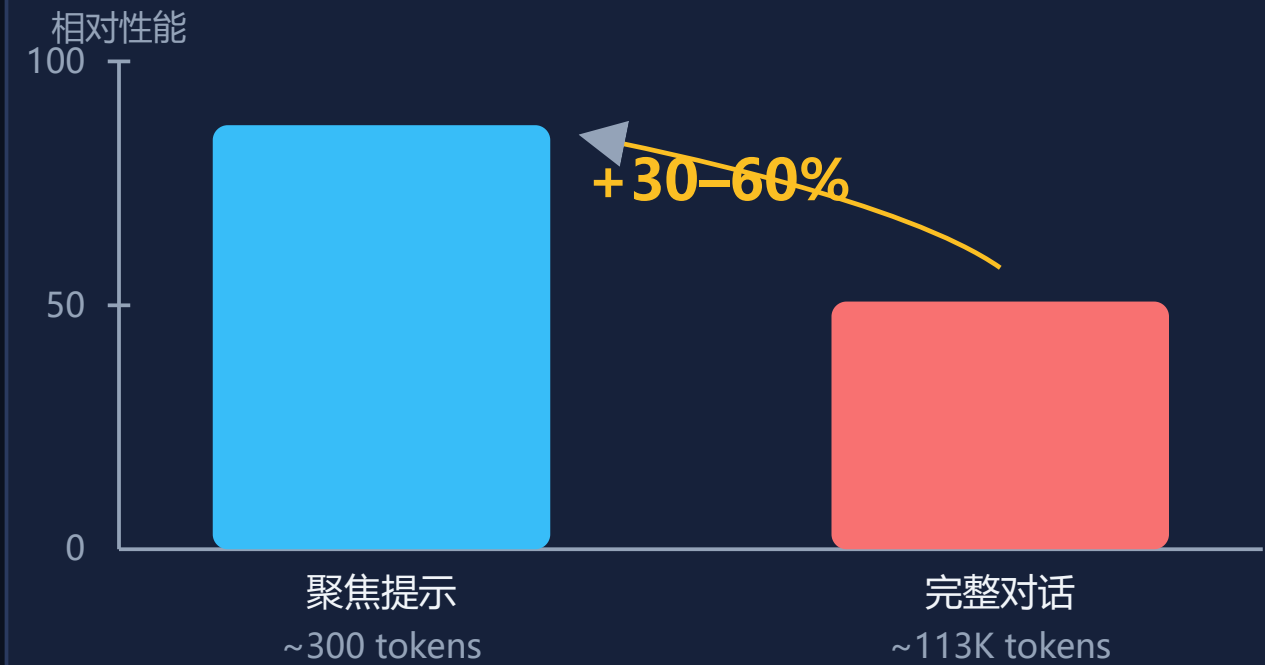


12 个宣称 128K+ 的模型  
11 个在 32K 跌破基线一半

## Context Rot：聚焦提示反而更强

实验：同一批任务只改变输入长度，统一测量多个模型——性能随 token 数增长全面下滑（context rot）。

读图：对照组中，~300 token 的聚焦提示比 ~113K token 的完整对话稳定强 30–60%。



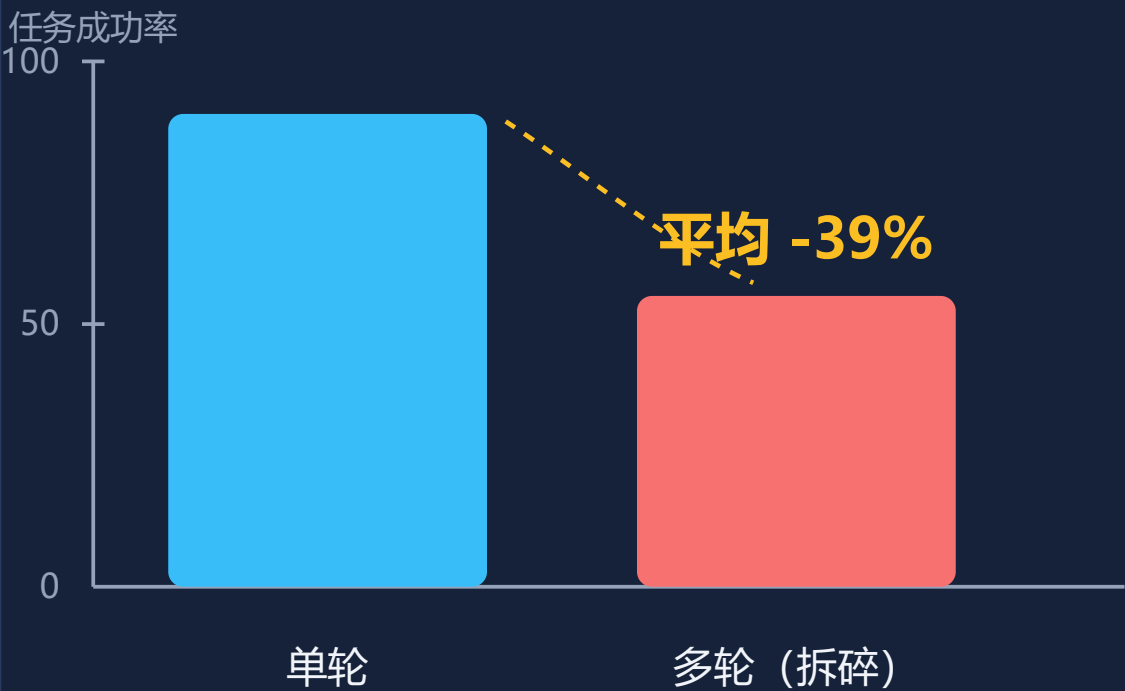
来源：Modarressi et al., NoLiMa, arXiv:2502.05167 (ICML 2025); Hong et al., Context Rot, Chroma Research, 2025-07

# 病灶四：多轮迷失——走错一步不回头

同一任务，单轮 vs 多轮，平均掉 39%

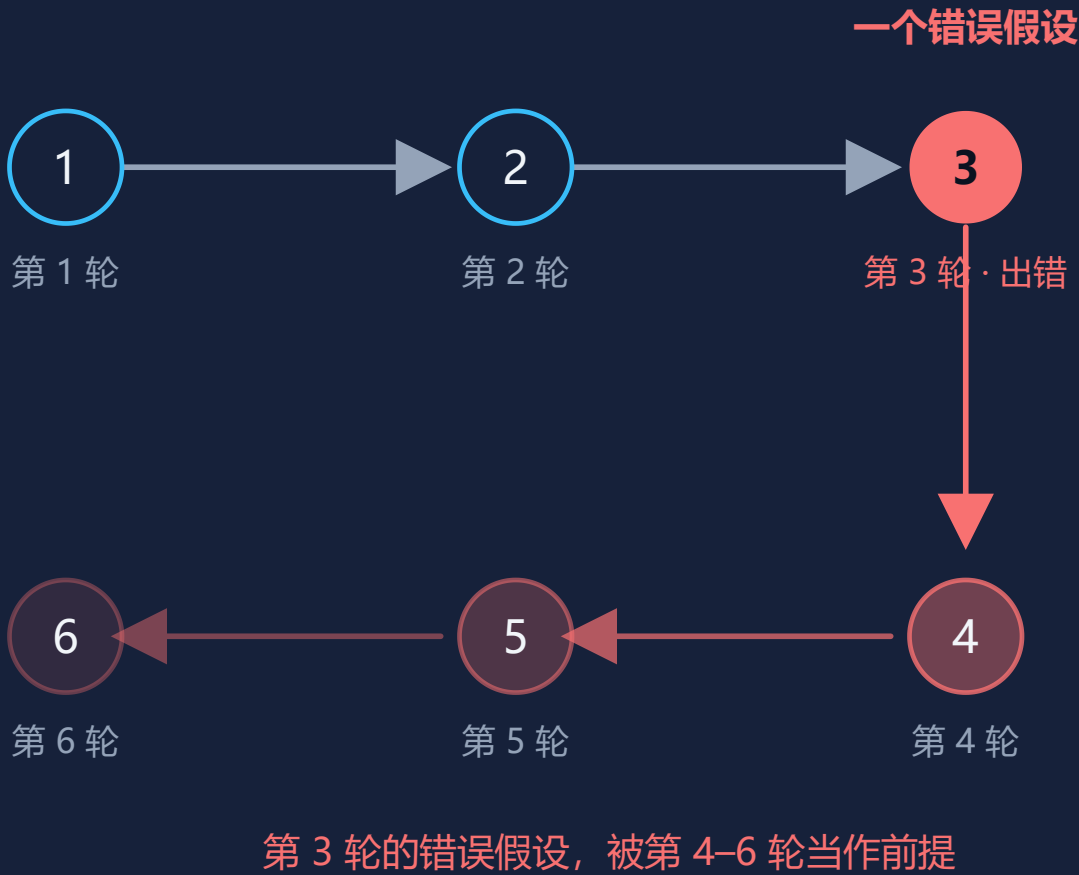
## 同一任务：单轮 vs 拆成多轮

实验：把同一个任务用两种方式交给模型——  
一次性说清（单轮）vs 分多轮逐步交代（多轮），  
任务本身完全一样；结果多轮平均掉约 39%。



## 错误一步，污染全程

机制：模型在早期轮次形成假设后死守不放——走错一步不回头，错误沉淀为后续所有轮次的前提。



来源：Laban et al., LLMs Get Lost In Multi-Turn Conversation, arXiv:2505.06120 (2025)

# 窗口容量 $\neq$ 窗口能力

退化是结构性的，等更大的模型没用——只能靠架构

本场详讲



## 切分

把长任务拆成小任务，  
每个 agent 只面对短上下文



## 隔离

子 agent 各自持有  
独立上下文，互不污染



## 压缩

把历史提炼成摘要，  
只保留高信息密度

本场详讲



## 外存

把状态写到上下文之外的磁盘，  
需要时再取回

病灶的解法，收敛为一个词

**context engineering**  
在推理全程动态维护最优 token 集合

**切分**

= 隔离上下文  
让每个 agent 待在聪明区间，  
注意力不被稀释  
事前 · P14–21

**落盘**

= 把状态搬出上下文  
进程会死，状态不死，  
断点可续  
事中 · P22–29

**校验**

= 给判断换干净上下文  
干活的人不验收，  
判断不被自己的产出带偏  
事后 · P30–38

第 2 部分 · 事前 · 12 分钟

# 事前：定契约，再动手

切分设计 + 「好」的机器可读定义

动手之前做两件事：第一，怎么切分任务，让每个 agent 都待在聪明区间里；  
第二，怎么在写第一行字之前，就把「什么叫做好」固化成机器可读的契约。



## 编排器-工人模式 (Anthropic 官方示意图)

中心 orchestrator 动态拆解任务 → worker 并行执行 → synthesizer 汇总结果



**Orchestrator**  
中心 agent 动态拆解、指派

**LLM Call 1-3 = Worker**  
各自持有独立上下文窗口

**Synthesizer**  
汇总结果输出

单个 agent 会越跑越笨，那就不用架构把它按回**聪明区间**——不指望它自律，是结构上**不给它变笨的机会**

Anthropic, Building effective agents, 2024-12-19 — 官方原图

# 7 角色泳道：每个角色在隔离上下文里完成小任务

Master 编排 —— 不亲自干活



数据准备、大纲、写作、图片、一致性、独立评审、验收——每个角色一个小任务，各自在隔离的上下文里跑；  
下载日志、字幕原文、图片候选这些又脏又长的材料根本不进写作 agent 的窗口——它拿到的只有大纲契约和要用的干净素材。

```
1 # Outline Contract: 幽灵般的智能：从 LLM 认知缺口到 AGI、ASI 与教育未来
3 ## 1. Editorial answer
5 The reader-facing answer is:
7 > Current large language models are extraordinarily effective statistical and cultural compressors, yet their
 competence is not the same as a durable, embodied, self-improving animal intelligence. The most important
 question is therefore not whether a model can produce an impressive answer, but whether it can learn from
 experience, assign credit across a long trajectory, create and verify novelty, operate reliably in the
 physical and social world, and remain understandable and controllable as autonomy scales.
9 The talk uses this distinction to connect five levels of reasoning:
11 1. Mechanism: pretraining, in-context learning, memory, reinforcement learning, credit assignment, and novelty.
12 2. Capability boundary: why current LLMs can look culturally intelligent while still exhibiting cognitive deficits.
13 3. System trajectory: how partial automation may progress toward AGI and ASI without requiring one discontinuous miracle.
14 4. Historical and deployment test: evolution, cumulative culture, self-driving, reliability, hidden operations, and economics.
15 5. Human consequence: education as a technical ramp to knowledge, then as human development...
17 The document must preserve the speaker's uncertainty. Forecasts and spoken numerical magnitudes remain
 attributed claims or scenarios, never precise empirical estimates.
```

写作之前先立契约：outline agent 先把「全局答案 + 五层推理 + 不确定性保留」写成机器可交接的契约  
契约内容：全局目录、术语表、符号表、章节边界、写作约定——**全部落盘成文件**，所有 writer 对齐同一份

```
1 # Writer 01 Handoff
3 ## Assigned sections
5 - Section 1: `总论：我们是在“建造动物”，还是在“召唤幽灵”？`
6 - Section 2: `LLM 的能力边界：为什么“会说”仍不等于“会做”`
7 - Written artifact: `section_01.tex`
9 ## Evidence ranges
11 - `00:00:52--00:03:44`: “the decade of agents” and the experience-based forecast caveat.
12 - `00:04:25--00:07:30`: task models, Atari/Universe, and the missing representation foundation of early agents.
13 - `00:07:53--00:13:01`: animal learning history, ghosts/spirits, and “crappy evolution”.
14 - `00:13:31--00:17:25`: pretraining as knowledge compression & algorithm discovery; in-context adaptation, mechanism uncertainty.
15 - `00:17:53--00:24:47`: weights versus KV cache, hazy recollection versus working memory, and the missing consolidation loop.
16 - `00:27:55--00:30:29`: nanochat reconstruction and hidden knowledge gaps.
17 - `00:30:43--00:31:37`: three coding collaboration modes.
18 - `00:31:37--00:35:09`: boilerplate, nanochat local constraints, DDP prior conflict, cursor bandwidth, and the Python-to-Rust
 constrained implementation example.
```

master 不亲自写：handoff 文件把**章节**、**证据时间区间**、**交付物文件名**一次交给 writer

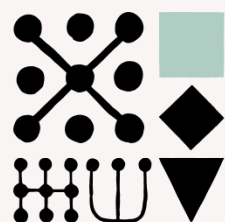
除了章节与证据区间，handoff 还把大纲契约里的三件事原样带给 writer——

**全局答案**：全书要回答的那个总问题；先传给每个 writer，各章才不会各写各的。

**五层推理**：从机制到人类后果的论证骨架；writer 按层落笔，章节才拼得回主线。

**不确定性保留**：预测和数字只能写成「发言人的说法」；防止 writer 把口头估计写成精确结论。

# 辩论：分，还是别分？（2025 年 6 月，同月两篇针锋相对）



## How we built our multi-agent research system

Published Jun 13, 2025

Our Research feature uses multiple Claude agents to explore complex topics more effectively. We share the engineering challenges and the lessons we learned from building this system.

VS

## Don't Build Multi-Agents

By Walden Yan

06.12.25

Cognition 的核心论据（Walden Yan · 2025-06-12，官网原文截图）：

We'll work our way up to the following principles:

1. Share context
2. Actions carry implicit decisions

Why think about principles?

## 「分！」

公开其 Research 产品的多 agent 架构——lead agent 规划分派，  
多个 subagent 各自持有独立上下文并行干活；  
内部评测：多 agent 系统比单个最强模型高出 **90.2%**。

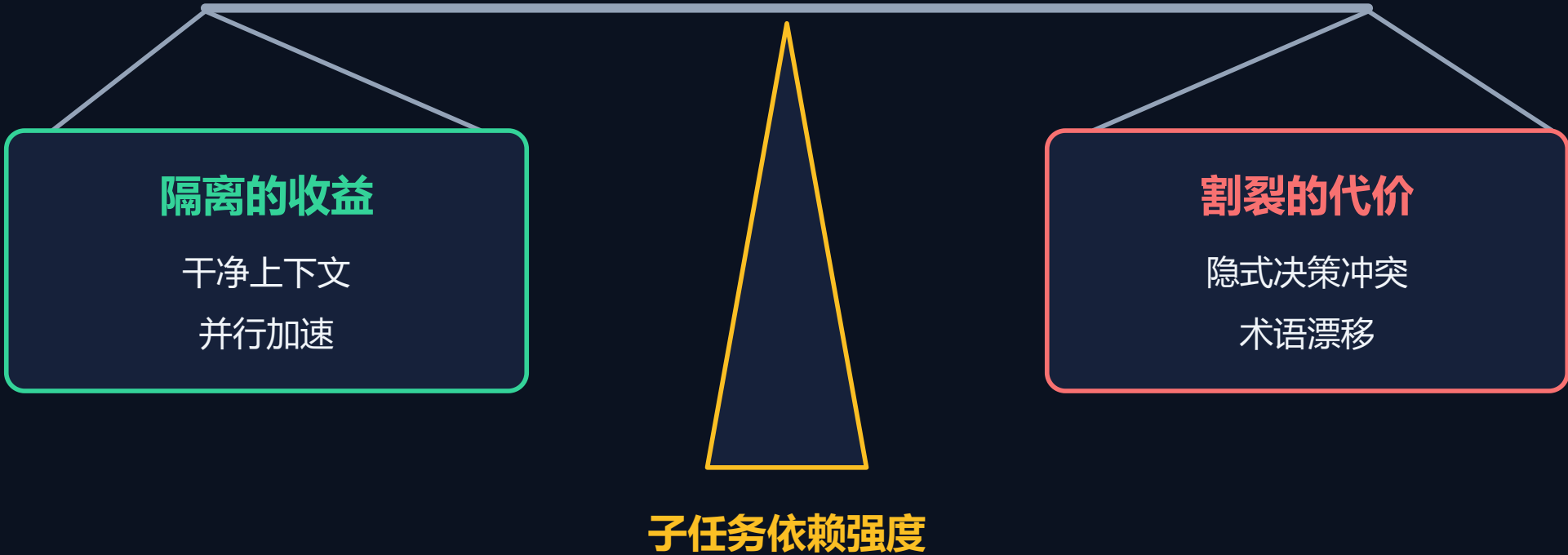
## 「别分！」

多个 agent 并行，每个都在做隐式决策——代码风格、接口假设、  
任务理解——互相看不见，合并时必然冲突；  
结论：默认单线程线性 agent；要共享就共享**完整轨迹**，别共享摘要。

左：Anthropic 《How we built our multi-agent research system》2025-06-13 · 右：Cognition 《Don't Build Multi-Agents》2025-06-12 — 官网标题截图拼版

# 共识：收益来自隔离，代价来自割裂

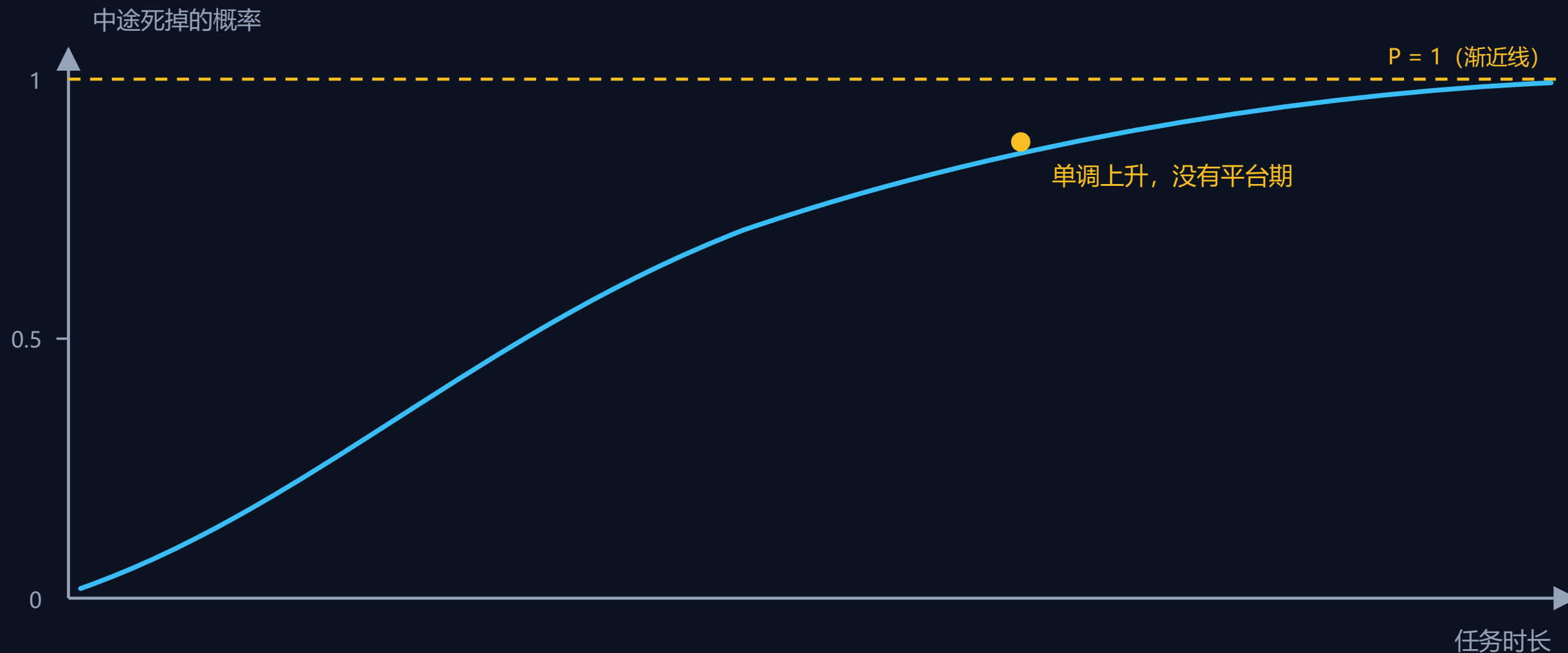
我们的总结



子 agent 互相看不见对方的隐式决策——共享得越少，合并越容易漂移。  
写 PDF 这件事，章与章是**弱依赖**：第三章不需要第二章的中间过程，只需要它的结论边界——所以适合分，多个 writer 并行。  
但术语是全文档**强依赖**：一个词前后译法不一致，全书就散了——所以不能分，必须共享：由大纲 agent 事先产出契约，所有人对齐同一份。  
**分不分，看依赖。**

## 药方二（预告）：任务越长，活到终点越难——状态要比进程命硬

任务越长，「中途死掉」的概率越接近 1



进程会崩、机器会重启、session 会断——这不是「会不会」的问题，是「早晚」的问题。  
所以可靠性设计的重心得挪一下：从「让进程不死」挪到「让状态不死」——进程可以随便死，  
只要状态在外面好好活着，就能从断点接着跑。这个思想有个正式名字：**durable execution**（持久执行）。

# 「好」的事前定义：rule catalog——标准先于产物存在

delivery-quality/v1/rule-catalog.v1.json · 第 7-11、123-136 行

```
7 "evaluation_policy": {
8 "blocking_only": true,
9 "fail_fast": false,
10 "report_all_failures": true
11 },
12 ...
123 {
124 "rule_id": "formula_information_gain",
125 "semantic_sha256": "34af3ebde7d1216ed489b50a60c9a4f47b7f07a08c7cdb43e4bd1091253f8981",
126 "scope": {
127 "artifact_kinds": ["integrated_text"],
128 "object_kinds": ["formula"],
129 "reader_facing_regions": ["body", "caption", "appendix"],
130 "trigger_condition": "A reader-facing formula or symbolic teaching model appears.",
131 "excluded_contexts": ["Source formulas preserved with attribution and necessary explanation."]
132 },
133 "requirement": "Every reader-facing formula adds usable quantitative, algorithmic, constraint,
 threshold, dependency, comparison, or decision value that justifies its cognitive cost.",
134 "violations": [
135 { "violation_id": "formula_has_no_information_gain", "observable_condition": "The formula merely
 renames nearby prose or lists variables without enabling new reasoning, estimation, comparison, or decision." }
136],
```

## 这张图在说什么

光有分工契约不够，动手之前还要定义清楚什么叫「好」：把「好」写成机器可读的规则目录（rule catalog）——每条规则带 ID、判定条件、严重级别；验收时独立 reviewer 对着目录逐项判，不凭感觉说「我觉得不错」。屏幕上这条是极端例子：formula information gain（公式信息增益）规定「什么时候才配用公式」——源材料本身有公式、概念本质上是数学的、或公式能表达 prose 表达不了的新约束，三种情况之外不许套公式。注意 **blocking\_only: true**——硬套公式不是「建议避免」，是验收直接拒收的阻断项。写作品味也能翻译成机器可执行的标准：标准先于产物存在，验收才有牙齿。

## 反例：伪公式长这样

给「创始人复盘创业心路」硬套一条「 $\text{成长速度} = \text{复盘频率} \times \text{迭代次数}^n$ 」——公式没有揭示任何新关系，只是把上一句话换了个写法：读者既不能用它推理，也不能用它决策。

workspace/Claude for Long\_Horizon Tasks ...\_20260728\_161214/review/acceptance/delivery\_glossary.json · 第 6-17 行

```
6 {
7 "english": "task horizon",
8 "chinese_primary": "任务时域",
9 "plain_language_boundary": "模型在需要人工重新介入前，能够持续完成的自主工作时长与复杂度；它不同于一次输入可容纳多少文本的上下文窗口。",
10 "related_terms": ["autonomous work", "long-horizon task"],
11 "opposed_terms": ["short interactive turn", "context window"],
12 "first_use_expected_location": "section_01.tex",
13 "body_display_strategy": "chinese_with_english_parenthetical",
14 "where_to_preserve_english": "body_parenthetical",
15 "required_after_first_use": "首次定义后统一使用“任务时域”；谈及上下文容量时必须使用“上下文窗口”。",
16 "forbidden_body_forms": ["任务视野", "上下文时域"]
17 }
```

让英文大模型写中文 PDF 有个非常具体的病——**术语漂移**：同一个英文术语，第三章这样译、第七章那样译；不是模型不懂，是它每次都在现场重新决定译法，靠叮嘱没用。解法是 **Delivery Glossary (交付术语表)**：大纲阶段就把每个英文术语钉死——固定中文主称、边界定义、首用位置、正文展示方式、英文原文保留位置、首用后的统一措辞；写作者无权现场决定译法，只有权查表。屏幕上 task horizon 钉死「任务时域」，边界定义写明「与上下文窗口不是一回事」；最妙的是 forbidden body forms (禁用写法) 里躺着「任务视野」「上下文时域」——历史上真出现过的错误译法，事先封死。

**好的术语契约，连错误译法都是事先封死的——把「写好中文」从品味问题变成工程问题。**

第 3 部分 · 事中 · 12 分钟

# 事中：隔离上下文，落盘每一步

worker 怎么干活 · 每一步怎么落盘 · 「不知道」一等状态



「好」的定义有了，术语也封死了——接下来，看执行。这一段只回答三个问题：

- ① 工人 agent 实际是怎么干活的
- ② 每一步的状态怎么落到磁盘上
- ③ 「我不知道结果」怎么成为一个正式的、被尊重的状态

workspace/Andrej Karpathy ...\_20260804\_232217/

└ main.tex

└ outline\_contract.md # 大纲契约（事前立约）

└ section\_01.tex # ← writer 直接产 TeX

└ section\_02.tex

└ section\_03.tex

└ section\_04.tex

└ source\_material\_handoff.md

└ source.en.srt

└ source.en.vtt

└ source.info.json

└ source.webp

└ source\_cover.png

└ source\_video.mkv 2.9 GB

└ 幽灵般的智能\_从 LLM 认知缺口到 AGI\_ASI 与教育未来.pdf

└ figures/

└ review/

└ 待删除/

└ work/ # ← 角色间靠 handoff 传话，展开见右窗 →

.../work/（13 个文件）

work/

└ outline\_agent\_handoff.md

└ writer\_01\_handoff.md

└ writer\_02\_handoff.md

└ writer\_03\_handoff.md

└ writer\_04\_handoff.md

└ figure\_01\_handoff.md

└ figure\_02\_handoff.md

└ figure\_03\_handoff.md

└ figure\_04\_handoff.md

└ segment\_01\_chapters\_01\_03.md

└ segment\_02\_chapters\_04\_06.md

└ segment\_03\_chapters\_07\_09.md

└ cover\_conversion.md

# 4 个 writer 各拿一份 handoff，互不进对方的上下文窗口。

writer 在干净上下文里直接产 section\_\*.tex；角色之间靠 work/ 下的 handoff 文件传话

# 多 agent 的 token 经济学 (Anthropic 博文数据段原截图)

Our internal evaluations show that multi-agent research systems excel especially for breadth-first queries that involve pursuing multiple independent directions simultaneously. We found that a multi-agent system with Claude Opus 4 as the lead agent and Claude Sonnet 4 subagents outperformed single-agent Claude Opus 4 by 90.2% on our internal research eval. For example, when asked to identify all the board members of the companies in the Information Technology S&P 500, the multi-agent system found the correct answers by decomposing this into tasks for subagents, while the single agent system failed to find the answer with slow, sequential searches.

Multi-agent systems work mainly because they help spend enough tokens to solve the problem. In our analysis, three factors explained 95% of the performance variance in the BrowseComp evaluation (which tests the ability of browsing agents to locate hard-to-find information). We found that token usage by itself explains 80% of the variance, with the number of tool calls and the model choice as the two other explanatory factors. This finding validates our architecture that distributes work across agents with separate context windows to add more capacity for parallel reasoning. The latest Claude models act as large efficiency multipliers on token use, as upgrading to Claude Sonnet 4 is a larger performance gain than doubling the token budget on Claude Sonnet 3.7. Multi-agent architectures effectively scale token usage for tasks that exceed the limits of single agents.

There is a downside: in practice, these architectures burn through tokens fast. In our data, agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats. For economic viability, multi-agent systems require tasks where the value of the task is high enough to pay for the increased performance. Further, some domains that require all agents to share the same context or involve many dependencies between agents are not a good fit for multi-agent systems today. For instance, most coding tasks involve fewer truly parallelizable tasks than research, and LLM agents are not yet great at coordinating and delegating to other agents in real time. We've found that multi-agent systems excel at valuable tasks that involve heavy parallelization, information that exceeds single context windows, and interfacing with numerous complex tools.

+90.2% 性能提升 (多 agent vs 单 agent)

Claude Opus 4 主 agent + Sonnet 4 子 agent, 内部研究评测

~15× token 消耗 (相对普通对话)

agent 约 4×, 多 agent 系统约 15× —— 用钱换可靠性

80% 性能差异由 token 用量解释

BrowseComp: 三因子合计解释 95% 的性能方差, token 用量独占 80%  
——换句话说, 性能提升很大程度是烧 token 烧出来的

结论很干脆: 多 agent 是**用钱换可靠性、换并行速度**——不是所有任务都配烧这个钱。  
什么任务值得? 三个特征: **广度优先** (要铺开查很多东西) · **可并行** (子任务能同时跑) · **弱依赖** (不需要彼此的中途状态)。  
我们的视频转 PDF 三条全中: 四章四个 writer 并行、章与章弱依赖。  
反过来, **强依赖、一步错步步错**的任务, 多 agent 只会放大混乱——那就别分。

Anthropic, How we built our multi-agent research system, 2025-06 — 数据段原截图 (左) + 数字提炼 (右)

# 落盘纪律：什么样的命令必须「留档运行」

D:\Project\video2pdf\newskill-kimi\待删除\long-running\（截取连续一段）

```
$ ls 待删除/long-running | wc -l
2717

$ ls 待删除/long-running
9QebvrrY3KY_acceptance_render_pages_02_20260728_192403_72e90835
9QebvrrY3KY_acceptance_render_pages_03_20260728_194953_20dcdaf4
9QebvrrY3KY_acceptance_repair01_pyramid_output_gate_20260728_191732_8f20ff6b
9QebvrrY3KY_acceptance_repair01_pyramid_output_gate_canonical_20260728_192038_754c9c76
9QebvrrY3KY_acceptance_repair01_pyramid_section04_evaluate_20260728_191352_3b62b50e
9QebvrrY3KY_acceptance_repair01_pyramid_section04_evaluate_canonical_20260728_191848_505e069a
9QebvrrY3KY_acceptance_repair01_pyramid_section04_evaluate_escalated_20260728_191529_52aaec3b
9QebvrrY3KY_acceptance_repair01_pyramid_section04_validate_20260728_191659_4bde01a4
9QebvrrY3KY_acceptance_repair01_pyramid_section04_validate_canonical_20260728_192005_ec0c232e
9QebvrrY3KY_acceptance_repair01_pyramid_section06_evaluate_20260728_191352_42ea5b08
9QebvrrY3KY_acceptance_repair01_pyramid_section06_validate_20260728_191659_5bb1eebd
9QebvrrY3KY_acceptance_skeleton_01_20260728_184511_96d212b5
9QebvrrY3KY_acceptance_skeleton_02_20260728_192426_e13720f0
9QebvrrY3KY_acceptance_validate_fail_01_20260728_190410_822968f2
9QebvrrY3KY_acceptance_validate_fail_02_20260728_194058_aad93882
9QebvrrY3KY_acceptance_validate_pass_03_20260728_200240_aca31f01
```

满足任何一条，就必须经过持久化运行器启动，不许直接在命令行里跑：

① 预计跑超过五分钟；② 可能活过当前 agent session（session 死了它还在跑）；③ 重跑很贵，或结果要拿去当验收证据。

运行器 fork 出一个脱离 session 的独立进程，把这次执行的一切落盘：命令是什么、状态是什么、退出码是多少、完整 stdout、完整 stderr。

## 2717 个不可变 run\_dir

每一次超过 5 分钟的执行都留下完整遗骸——包括 fail\_01、fail\_02 之后才出现的 pass\_03

# 解剖一个 run\_dir: 证据不在 session 里, 在磁盘上

待删除/long-running/<run\_dir>/

9QebvrrY3KY\_acceptance\_criteria\_validate\_20260728\_184345\_fa37b8eb/

└ .status.lock	1 B
└ command.json	589 B
└ command.log	99 B
└ exit-code.txt	2 B
└ status.json	785 B
└ stderr.log	0 B
└ stdout.log	87 B
└ supervisor-identity.json	263 B

# exit-code.txt 里是真实退出码;

status.json 全文见右窗 →

session 死了, 换个人用

list → show → wait 接管观测。

status.json · 全文 (折行展示, 字段与值原样)

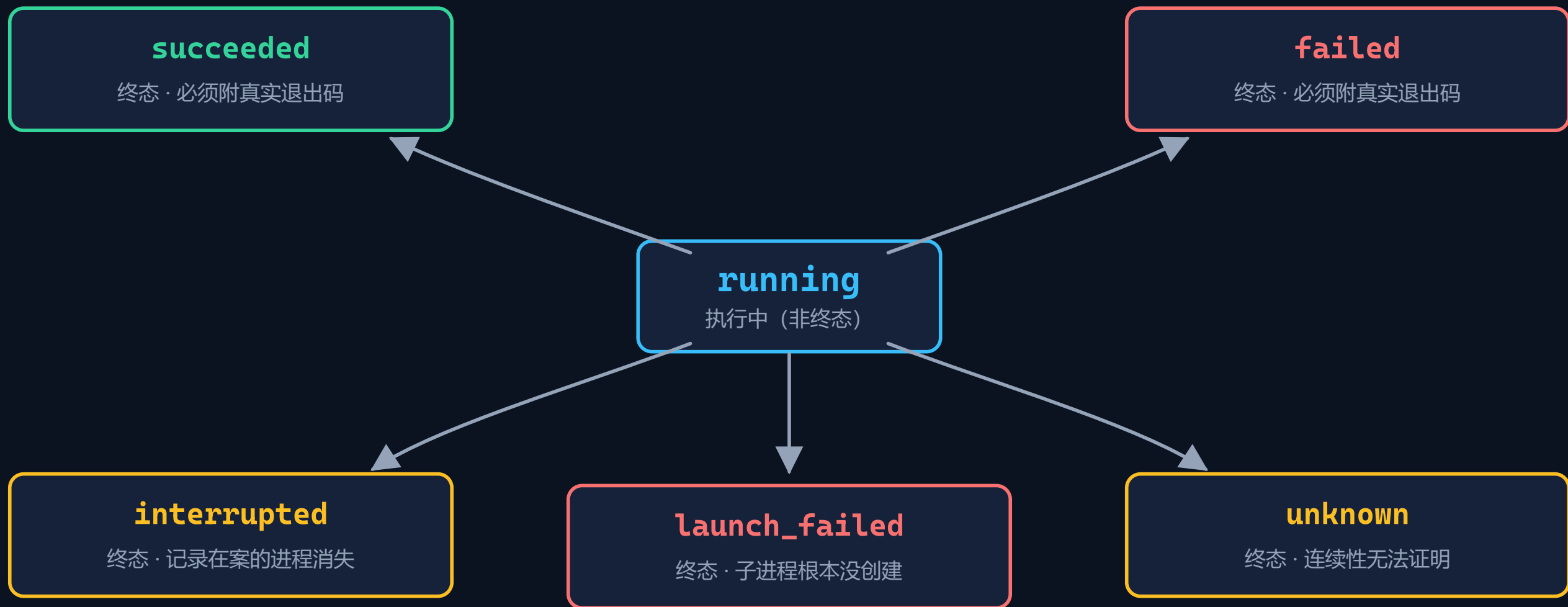
```
{
 "child_pid": 51976,
 "elapsed_seconds": 1.062,
 "exit_code": 0,
 "finished_at": "2026-07-28T18:43:49.056+08:00",
 "heartbeat_at": "2026-07-28T18:43:49.060+08:00",
 "latest_output_at": "2026-07-28T18:43:48.871+08:00",
 "log_sizes": {"merged": 99, "stderr": 0, "stdout": 87},
 "run_id": "fa37b8eb-a4e7-4763-a165-8fa442ab0f98",
 "schema_name": "persisted-command-status",
 "schema_version": "1.0.0",
 "security": {
 "acceptance_evidence_eligible": true,
 "classification": "no_secret_detected"
 },
 "started_at": "2026-07-28T18:43:45.701+08:00",
 "state": "succeeded",
 "supervisor_identity": {"pid": 45076,
 "process_creation_identity": "windows-filetime:134297090258119624"},
 "supervisor_pid": 45076,
 "target_identity": {"pid": 51976,
 "process_creation_identity": "windows-filetime:134297090280815848"},
 "updated_at": "2026-07-28T18:43:49.060+08:00"
}
```

五件套一目了然: `command.json` 记完整命令行和工作目录; `status.json` 是状态快照——state: succeeded、exit\_code: 0, 还有 heartbeat\_at, 运行期间每 29 秒跳一次心跳; `exit-code.txt` 单独存退出码; `stdout.log` / `stderr.log` 完整输出, 一个字节不截断。

场景: 半夜 agent session 挂了, 命令还在跑; 第二天开新 session, 三步接管——`list` 列出所有持久化运行目录, `show` 看完整快照 (还在跑、心跳是三分钟前的), `wait` 挂在那儿等终态。全程不需要原 session 活着——证据不在 session 里, 在磁盘上。

细节: status.json 记 supervisor 和 target 双重进程身份, Windows 上连文件时间戳都记了——防止 PID 被系统复用后误判「进程还活着」。

# 状态语义：把「不知道」变成一等公民



落盘最怕「看起来跑完了」。五种状态：**launch\_failed**——压根没启动，没有子进程；**running**——在跑，有心跳为证；终态 **succeeded** / **failed** 不许随便用——必须附上真实退出码：进程真退出了、退出码真拿到了，才配用。如果 session 死时进程还在跑、后来进程也没了——没证据说成了，也没证据说败了，那就诚实一点：**interrupted**（记录在案的活进程消失）、**unknown**（连续性无法证明）——这两个也是合法终态、状态机的一等公民。旁边代码视图的终态判定读的是退出码文件和进程身份，不读日志里有没有「完成」两个字——agent 可以说「我觉得跑完了」，状态机只认证据。

**agent 会下线，证据不下线；把「不知道」变成一等公民，就是对治汇报乐观偏差——宁可说不知道，不许装知道。**

src/video2pdf\_persisted\_command/cli.py · 终态判定 (1366-1378)

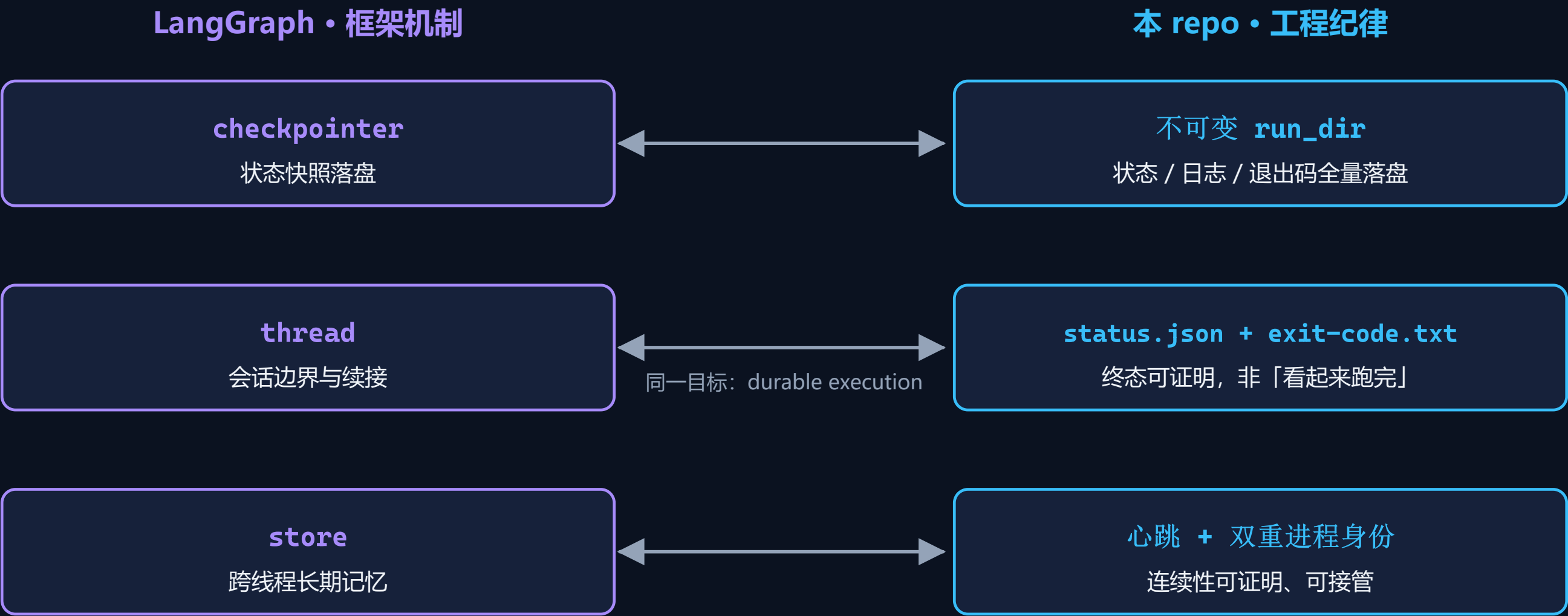
succeeded / failed 必须附真实退出码

```
1366 exit_code = process.wait()
1367 for reader in readers:
1368 reader.join()
1369 _write_text_atomic(run_dir / "exit-code.txt", f"{exit_code}\n")
1370 terminal_state = (
1371 "failed"
1372 if log_persistence_failed
1373 else (
1374 "succeeded"
1375 if exit_code in command_record["accepted_exit_codes"]
1376 else "failed"
1377)
1378)
```

不允许「看起来跑完了」——**interrupted / unknown** 同样是合法终态，agent 会下线，证据不下线

日志持久化失败也强制判 failed：证据链断了，结果再好也不算数

# 不靠框架也能 durable



概念对照: LangGraph Persistence · Temporal Durable Execution · Anthropic「穷人的 durable execution」 (2025-11-26)

workspace/Andrej Karpathy \_ We\_re summoning ghosts...\_20260804\_232217/待删除/

```
$ ls 待删除/

├─ figure_candidates/ # 图片候选帧，全程保留
├─ latex-build/ # LaTeX 编译中间产物
├─ pyramid-evaluator/ # 结构校验临时文件

├─ format_inventory.log
├─ metadata_thumbnail.log
├─ subtitle_en_srt.log
├─ subtitle_en_vtt.log
└─ subtitle_inventory.log

workflow纪律：中间产物一律暂存进 待删除/，永不自动删除——
删除是人类的决定，不是 agent 的决定。

（按脱敏规则，该目录下的 cookie 文件名不在此展示）
```

不删，只暂存进 待删除/——agent 没有删除权，删除留给人类

workspace/AI Security After Codex and Claude Code ...\_20260727\_144154/review/consistency/consistency\_report.md · 第 3-7、49-71 行

3

## Decision

5

**\*\*Overall: FAIL\*\***

7

The final-artifact structure is internally sound: `main.tex` contains one exact inlined copy of each root  
`section\_01.tex` through `section\_07.tex`; current source fingerprints match all nine Pyramid reports; ...  
:  
49 | Term | First-use and preservation evidence | Later-use and chapter consistency | Result |  
50 | --- | --- | --- | --- |  
51 | AI security / AI 系统安全 | `section\_01.tex:4` gives `AI 系统安全 (AI security)` in the contracted  
opening location. | Stable Chinese at `section\_01.tex:7,11`; no later English repetition. | PASS |  
56 | red teaming / 红队测试 | `section\_01.tex:68` uses `红队测试` before the contracted bilingual  
definition at `section\_02.tex:22`. | Later prose is stable, though headings also shorten the concept to `红队`. | **\*\*FAIL C1\*\*** |  
57 | automated red teaming / 自动化红队测试 | Contracted bilingual definition appears at `section\_02.tex:37`.  
The reader first encounters the shortened `自动化红队` in `section\_02.tex:1`, before that definition. |  
Full primary name is stable in sections 06 and 07; the chapter title remains a pre-definition variant. | **\*\*FAIL C1\*\*** |  
58 | out-of-distribution / 分布外 | `section\_02.tex:1` uses `分布外` before the contracted bilingual  
definition at `section\_02.tex:37`. | Later uses in 06/07 keep the Chinese concept, no English repetition. | **\*\*FAIL C1\*\*** |  
69 | grey swan / 灰天鹅事件 | Contracted bilingual definition appears at `section\_06.tex:63`; the title  
`section\_06.tex:1` first exposes the shortened `灰天鹅`. | `section\_06.tex:77` defines the term a second  
time. Later synthesis uses `灰天鹅风险`, another shortened form. | **\*\*FAIL C1, C2\*\*** |  
71 The same failing reader order appears in `main.tex`: early occurrences at lines 174, 180, 258, ... precede bilingual definitions ...

① 这是什么 | 第二条执行纪律：写完所有章节后，Consistency agent 拿事前定的术语表（Delivery Glossary）逐章逐项核对五件事  
首次使用的措辞对不对 · 后续用法稳不稳定 · 源英文保留在该留的位置没有 · 正文展示策略前后一致吗 · 章与章之间术语打架吗

② FAIL 在哪 | Overall: **FAIL**——真挂了：结构检查全 PASS，卡在术语契约，两类违规共 **10 条**  
**C1 首用顺序违规（主因）**：读者在契约规定的双语定义之前就先遇到术语或简称——  
例：「红队测试」在 section\_01:68 先被使用，契约定义「红队测试 (red teaming)」迟至 section\_02:22——  
定义迟到，首用失效；章节标题抢先用了简称「灰天鹅」，正文后面才定义「灰天鹅事件 (grey swan)」。  
**C2 二次定义**：同一术语被定义了两遍（如 reward hacking、grey swan、agent-native identity）。

③ 处置 | 全部拦在交付前，打回修改  
真挂了的 **FAIL** 报告比 pass 更有  
说服力：术语首用顺序逐项核对，  
**10 条违规** (FAIL C1/C2)  
被拦在交付前。

# 事后：干活的人，交叉评审

回扣开场：那个视频就是被这段的制度拦下的

好，活干完了。事前定了契约，事中隔离了上下文、落盘了每一步——  
但长程任务最危险的一步，现在才刚开始：谁来判它「好」？  
还记得开场那个讲长程任务的视频吗？它挂了两次，  
拦下它的不是某个人的火眼金睛，正是接下来这九页要讲的验收制度。  
这一段我们回答一个问题：验收，本质上是一场权力设计。



# 自评偏差：两点成因打头

为什么「干活的人，交叉评审」——不是纪律洁癖，是机制使然

## 成因 ①

### 上下文 · 历史上下文干扰

干活的那个上下文里，装着它全程的辩护痕迹——它为什么这么写、这么改，每一步决策都有理由留在上下文里。等它回头验收时，它看到的不是是一份中性的稿子，而是一份「每一步都有我亲批理由」的稿子。它不是在验收，是在重读自己的辩护词。

## 成因 ②

### 模型推理 · 推理路径的自我肯定

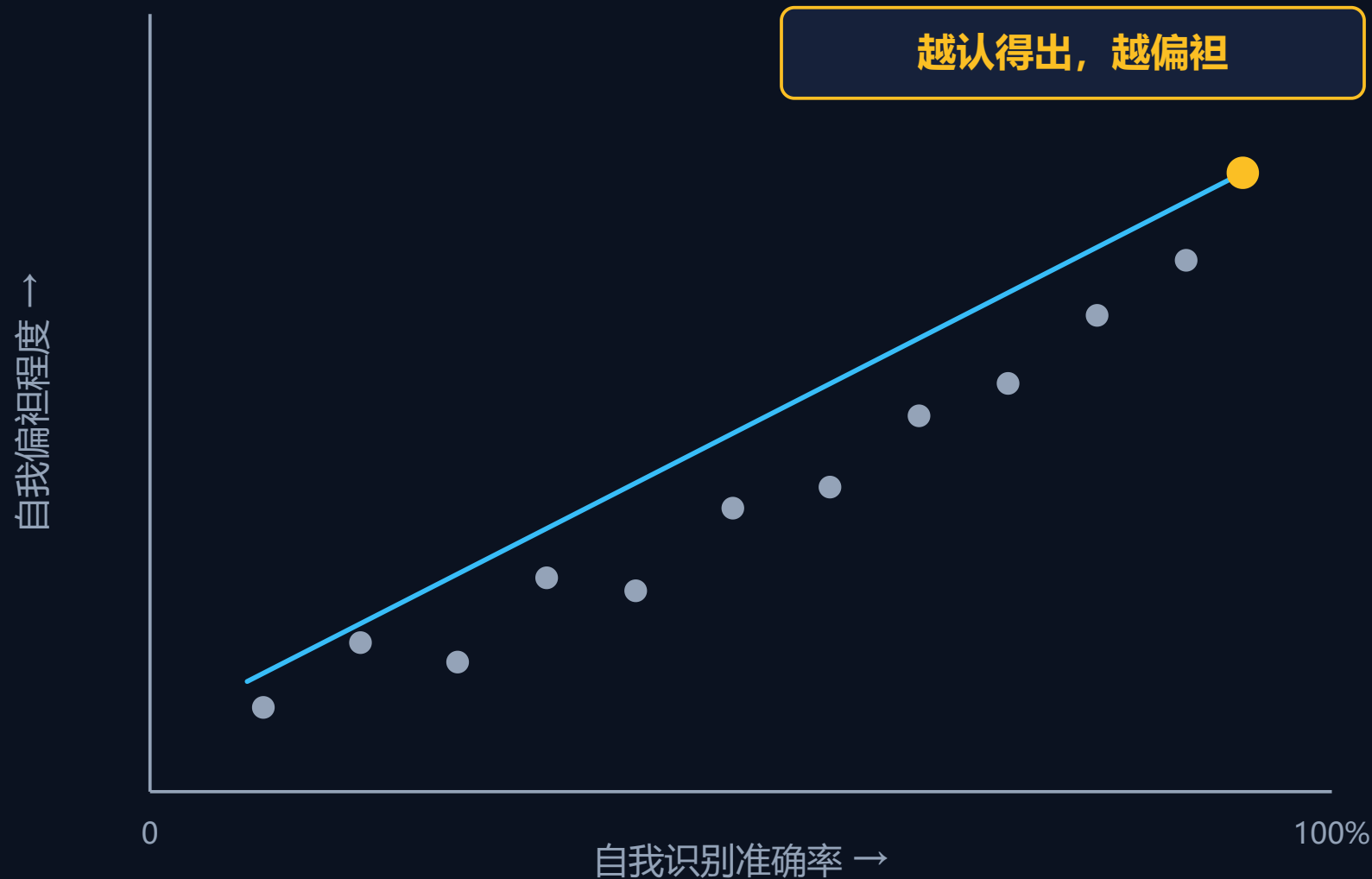
大模型的本质是 `predict next token`（预测下一个词）。它对自己刚生成的内容，预测起来天然顺畅——因为那就是按它自己的分布生成的。每一个词都长在它的审美上。「读着顺」和「质量高」在它内部是同一个信号，根本分不开。你让它挑错，等于让它找「哪里不顺」——可通篇都是顺着它的路子来的，它找不到不顺的地方。

## 自评必然偏肯定

不是态度问题，是上下文与生成机制共同决定的结构性偏差

现象层成因 · 机制证据见下两页：Self-Preference Bias (arXiv:2404.13076 / 2410.21819) · 自纠失败 (arXiv:2310.01798 / 2507.02778)

# 机制证据：Self-Preference Bias —— 越认得出自己，越偏袒自己



图源：Panickssery, Bowman & Feng (NeurIPS 2024)

## 两篇论文：认出就偏袒，匿名也没用

第一篇 (NeurIPS 2024)：Panickssery、Bowman 和 Feng。他们让 LLM 当评审，评一堆回答——其中混着它自己写的。发现一：评审器能以显著高于随机的准确率认出「这是我自己写的」，模型对自己的笔迹有感觉；发现二，也更关键——自我识别能力与自我偏袒程度呈线性相关：越认得出，越偏袒。认出是自己写的，分数就往上抬；认得越准，抬得越狠。你可能会想：那简单，匿名评审不就行了？不告诉它这是谁写的。第二篇，Wataoka 等人 (2024) 把这条路也堵上了：即使没显式认出是自己写的，偏袒依然存在。匿名没用。偏袒从哪来？根因之一是熟悉度——自己的输出，对它自己来说 perplexity (困惑度) 最低：预测最省力、「最顺」。「读起来顺」被它误当成了「质量高」。回头看上一页第二个成因——推理路径自我肯定，跟 perplexity 机理互为印证：工程侧看到的现象，和学术侧测到的机制，指向同一件事。这不是态度问题，是预测机制层面的结构性偏差。态度可以教育，结构只能绕行。

结构性的问题，只能靠结构来解决，不能靠自觉。

Panickssery et al., arXiv:2404.13076 (NeurIPS 2024)  
Wataoka et al., arXiv:2410.21819 (2024)

# 自纠失败：挑得了别人的错，挑不了自己的

纠正外部输入的错误

有效 ✓ 明显提升

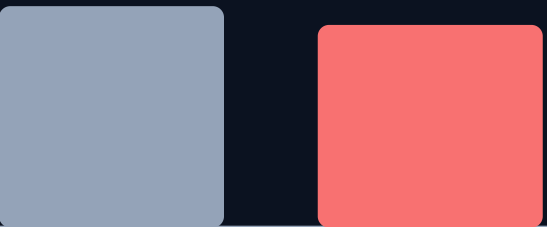


纠正前

纠正后

纠正自己输出的错误

基本无效 ✗ 甚至更差



纠正前

纠正后

到这儿，总有人想走一条偷懒路线：好，评分不让它自己评了，那我让它自己检查、自己改，总行吧？写完自己过一遍，发现错误自己修——听起来多省事。学术界的回答是：不行，这条路已经被封死了。Google DeepMind 和 UIUC 的一篇 ICLR 2024 论文，标题就很直白——《大语言模型尚不能自我纠正推理》。结论：在没有外部反馈的情况下，让 LLM 自我纠正推理错误，性能基本不提升，甚至更差。那以前那些「自纠有效」的报道呢？论文也回答了：大多偷偷用了隐藏的外部信息，比如拿了标准答案当线索。那不叫自纠，那叫作弊。

2025 年的 Self-Correction Bench 刻画得更精准：模型能纠正外部输入的错误，却纠正不了自己输出的错误。他们叫它「自纠盲区」。

同一个错误，别人犯的它一眼看穿，自己犯的它视而不见。这句话值得单独读一遍：纠得了别人的，纠不了自己的。

跟上一页合起来就是两刀——评分时偏袒自己，修改时改不动自己。所以检查能力必须来自生成上下文之外——新的上下文、独立的 agent、工具、或者人。这不是工程偏好，是有论文撑腰的硬约束。

「检查能力必须来自生成上下文之外」

Huang et al., *Large Language Models Cannot Self-Correct Reasoning Yet*, arXiv:2310.01798 (ICLR 2024)  
*Self-Correction Bench*, arXiv:2507.02778 (2025)

# 换个 LLM 当裁判？裁判自己也带病



## 位置偏差

Position Bias

交换两个答案的展示顺序，  
裁判结论**就可能翻转**

判的不是对错，是先后



## 冗长偏差

Verbosity Bias

更长的回答更容易被判好，  
**不管内容是否更正确**

字数成了质量的代理指标



## 自我增强偏差

Self-Enhancement Bias

裁判偏爱与自己同源的输出——  
**自评偏差在裁判席重演**

换裁判 ≠ 换机制

结论：裁判必须独立，且裁判的结论也必须被模型之外的制度约束，不能只靠模型内生的能力

Zheng et al., *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, arXiv:2306.05685 (NeurIPS 2023)

# 权力分离: Generator ≠ Verifier ≠ Publisher



Anthropic CitationAgent (引用核查代理) · Claude Code 独立评审子代理 · 本 repo Acceptance Reviewer (验收评审员)  
同一模式的三个实例: 评审者 ≠ 作者, 在 agent 时代的必然重演

诚实备注: 该工程模式暂无公认锚定论文 (证据等级: 厂商实践 + 本 repo 制度)

review/ 目录树（节选）

```
review/
├─ independent_review/ # 独立评审 agent: 对照原始字幕查漏，迭代到满意为止
│ └─ independent_content_review.md
│ └─ independent_content_review_round_02.md # 第 2 轮
├─ acceptance/
│ └─ control-store/ # 已建（空）：v2 执行记录即将落位
│ └─ delivery_glossary.json
│ └─ rendered_pages/ # 36 页逐页渲染 PNG：验收读集已就位
│ └─ page_0001.png
│ └─ page_0002.png
│ └─ ...
│ └─ page_0036.png
├─ consistency/
├─ latex/
└─ pyramid/
```

验收评审员：只读固定读集（路径加 SHA 指纹）· 只写判断补丁· 无发布权——制度已上线、首单在路上

第一道岗 · 独立评审 agent

管内容完整性。第一版 PDF 出来后，另起一个全新的 agent——「另起」意味着全新上下文，不是写作 agent 换个帽子接着干。它对照原始字幕逐条查漏——对照的是源材料，不是写作者的记忆。工作流必须迭代到它判定「内容足够完整」为止：它提一轮意见、写作者回一句「已改」不算完；它不满意，循环就不停。

第二道岗 · Acceptance Reviewer（验收评审员）

最终验收员，管交付质量。权限被钉死在契约里，四条：一、只读固定读集——每个文件带路径加 SHA 指纹，读集之外一概碰不到，想顺手翻别的材料都不行；二、只写一份东西：判断补丁（Judgment Patch）；三、没有报告发布权——补丁由协调方提交、物化，验收报告才算数；四、中途断了能恢复，不用从头再来。判断归它，发布归别人——上一页那张三权分立图，在这儿带文件路径。

等待窗口 · 超时不等于失败

我们给它一小时等待窗口，明文规定：超时不等于失败。评审是慢功夫，等不及掐死它、换个跑得快但看得糙的，就是在制度上奖励敷衍。

回扣开场

那个讲长程任务的视频，就是被这两道岗拦下的：挂两次、修两次、第三次放行。新制度已上线，v2 验收的第一单，正在路上。

```
rendered_pages/ - 27 files

page_0001.png
page_0002.png
page_0003.png
page_0004.png
page_0005.png
page_0006.png
... (page_0007 - page_0024)
page_0025.png
page_0026.png
page_0027.png

page_0001 ... page_0027 · 共 27 张
```

```
acceptance_report.json - visual_scan_evidence (schema_version 1.0)

{
 "visual_scan_evidence": {
 "pages_checked": [← 27 条，一页一条
 { "page": 1,
 "rendered_page_image": "review/acceptance/rendered_pages/page_0001.png",
 "status": "pass", "failures": [] },
 { "page": 2, ... "status": "pass", "failures": [] },
 ... page 3 - page 26: 同构，全部 "pass" ...
 { "page": 27,
 "rendered_page_image": "review/acceptance/rendered_pages/page_0027.png",
 "status": "pass", "failures": [] }
]
 }
}
```

制度定好了，还得防执行的人偷工减料。最典型的偷懒：视觉验收。一本 27 页的 PDF，每页都要渲染成图、逐页检查。时间紧的时候，agent 天然想抽几页看看，或拼张缩略图总览就算看过。我们的政策把这条路逐一点名封死：必须逐页检查全部页面——缩略图、抽样、拼版总览只配当导航，不能当下判依据；每一页都要在验收报告里留一条对应的检查记录；做不完就不许交付，不许缩水证据集换一个 pass。

但这里有一个我必须诚实交代的边界：机器门禁能证明「每一页的图都在」，证明不了「评审员真的看了每一页」。所以政策明文规定：基于缩减输入拿到的 pass，是无效证据，照样拦下。

机械校验有它的边界，承认这个边界，制度才可信。

逐页政策：27 张 PNG ↔ 27 条 pages\_checked，缺一页即无效 (v1 时代报告)

workspace/Claude for Long\_Horizon Tasks \_ Lance Martin\_ Anthropic\_20260728\_161214/review/acceptance/delivery\_guard\_report.json

```
{
 "status": "pass",
 "checked_at": "2026-07-28T20:03:50+08:00",
 "stage": "accepted",
 "validated_by": "delivery_guard.py",
 "acceptance_report_status": "pass",
 "checked_conditions": [
 { "condition": "target_resolved", "status": "pass" },
 { "condition": "allowed_artifacts_manifest_loaded", "status": "pass" },
 { "condition": "final_pdf_in_manifest", "status": "pass" },
 ... 共 7 项检查, 全部 "pass" ...
 { "condition": "artifact_fingerprints_current", "status": "pass" }
],
 "blocking_message": null
}
```

**视图一 · 这份报告怎么读**  
最后一道，纯机械守卫：Delivery Guard  
(交付守卫)。状态 **pass**，七项检查逐项打勾：  
交付目标解析、最终 PDF 在清单、编译来源  
新鲜、验收报告当前有效、逐页证据齐全、  
产物 SHA-256 指纹一致、全局门禁权威唯一。  
它只认新鲜的 **pass**——验完你又改了文件？  
**指纹对不上，重来。**编译报告想冒充验收报告？  
**明文规定，不行。**

干活的人不评审，评审的人不发布，发布之前过机器。

七项机械校验全 pass、blocking\_message = null —— 才允许交付

**视图二 · 两道兜底**  
再翻视图二，两道兜底。第一兜：失败有界，  
尝试上限写死就是 **3**，修三次还不过，留档、  
写简报、升级人工——**不许无限重试。**  
第二兜，**Stop hook** (停止钩子)：session  
停下的最后一厘米，自动补跑一次守卫，  
**没过就不许走。**  
回头看那句「自纠需要外部反馈」——这套  
门禁，就是外部反馈的制度化。

.agents/skills/final-delivery-acceptance/scripts/delivery\_guard.py · 233-238

233

def \_validate\_attempt\_limit(value: Any) → int:

234

if not isinstance(value, int) or isinstance(value, bool):

235

raise GuardError("delivery\_target.attempt\_limit must be an integer")

236

if value ≠ 3:

237

raise GuardError("delivery\_target.attempt\_limit must be 3")

238

return value

失败有界：写死在代码里，不是 prompt 约定

三次验收失败 → **blocked** + manual\_repair\_brief.md, 交还人类

机器守卫拒绝任何 attempt\_limit ≠ 3 的交付目标——「标准也是代码」

# 带走一张检查清单

长程任务可靠性清单 · 可截图带走

## 事前 · 定契约

- ✓ **分工契约**  
编排器-工人模式 · 工人独立上下文
- ✓ **机器可读的「好」**  
规则目录 · 连公式都有拒收规则
- ✓ **术语封死**  
交付术语表 · 连错误译法都禁用

## 事中 · 隔离 + 落盘

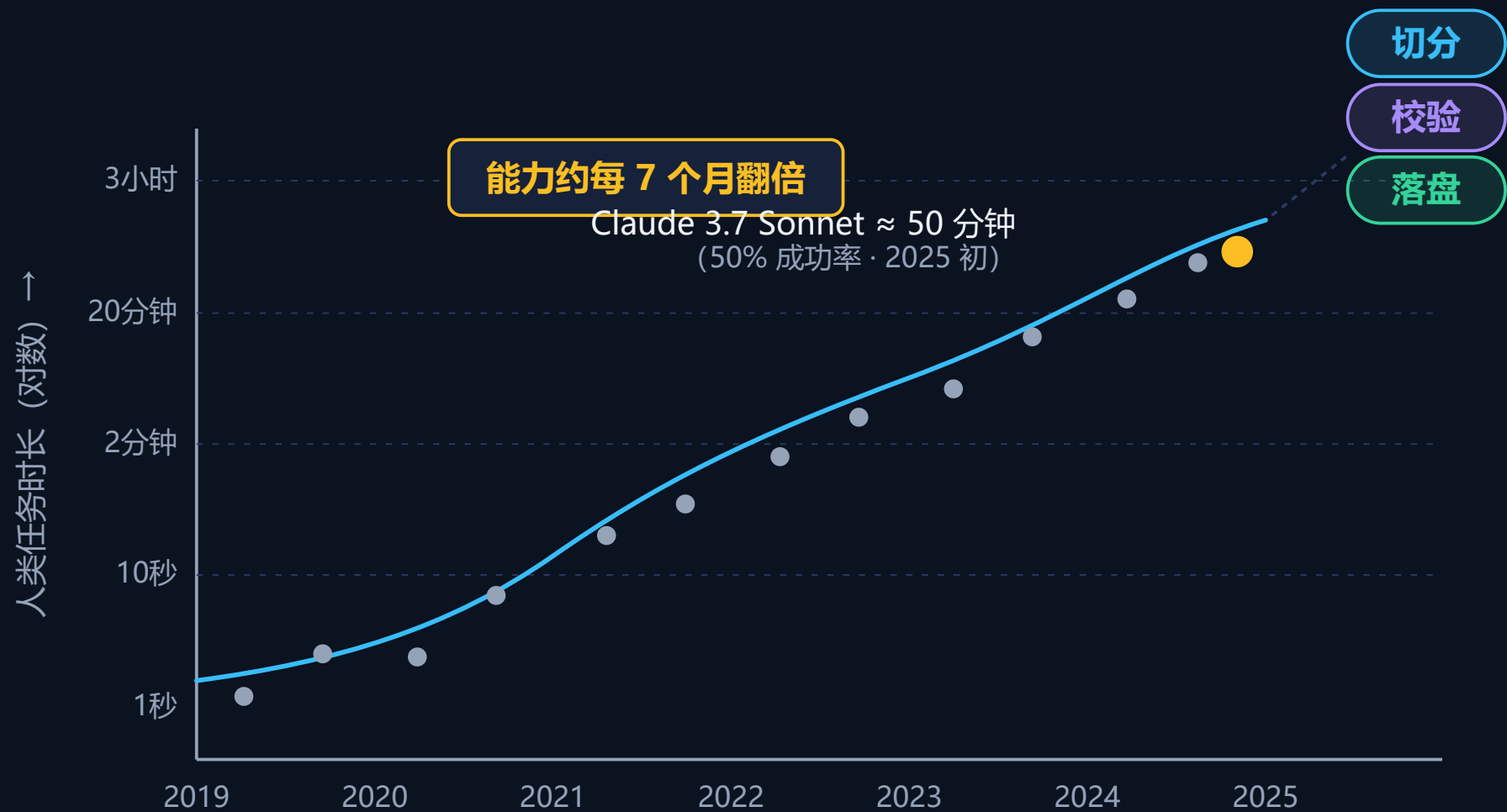
- ✓ **聪明区间**  
切分把小任务按回短上下文
- ✓ **每步落盘**  
持久化运行目录不可变 · 状态不死
- ✓ **「不知道」一等状态**  
「中断」「未知」也是合法终态

## 事后 · 独立验收

- ✓ **验收者独立**  
生产者 ≠ 校验者 ≠ 发布者
- ✓ **标准可证伪**  
公式收益门缺证据 · 机械拒收
- ✓ **证据新鲜有指纹**  
SHA-256 指纹 + 新鲜度校验
- ✓ **失败有界**  
尝试上限写死为 3 · 三次不过即锁定

切分保聪明 · 落盘保连续 · 校验保正确

# 回扣：曲线每 7 个月翻倍，靠可靠性工程在你的系统里兑现



还记得开场那条曲线吗？人类五十分钟的任务，前沿模型只有一半成功率，而这个时长**每七个月翻一倍**。今天讲的一切——**切分、落盘、独立验收**——不是为了让曲线更陡，是为了让这条曲线在你的系统里真的兑现。能力翻倍是模型厂商的事，把翻倍的能力变成翻倍的**可靠交付**，是我们的事。即：提升模型的**可靠性**，尽量让每次执行都发挥**模型能力的上限**。

Kwa et al. (METR), *Measuring AI Ability to Complete Long Tasks*, arXiv:2503.14499 (2025) · 曲线为示意图重现

旁白

最后一句旁白。这套 **harness** (脚手架) 自己也是一个**长程任务**——它跨了好几个月、几十个 agent session 才长成今天这样。为了防止它自己**决策失忆**，我们给它记了**六十四条架构决策记录, ADR**。

ADR = Architecture Decision Record

(架构决策记录)：每条记下「当时为什么这么定」，让后来的 session 不重辩已决问题（展开见附录 B7）。

给开发 harness 的 agent 用的 harness，**就是这本账**。

开场那个挂两次的视频：**第三次过了，证据链完整**

开放问题

你的任务里，哪一步最值得  
先落盘 / 先独立验收？

---

切分的代价（上下文割裂），  
你打算怎么付？

没有标准答案——但值得在下一个任务启动之前就想清楚

# 延伸阅读 · 今日引用

## 深入读物

### OpenAI 《A Practical Guide to Building Agents》

guardrails 章：把「好」写成护栏，而不是愿望  
openai.com · 2025

### HumanLayer 《12-Factor Agents》

重点推荐

小 agent · 窄上下文 · 状态外置 · 错误即上下文  
github.com/humanlayer/12-factor-agents

两本分别覆盖「护栏怎么定」与「工程纪律怎么立」，  
与今天三条主线（切分 / 落盘 / 校验）一一对应。

## 今日引用

### 论文

arXiv:2503.14499	METR 时间地平线	arXiv:2404.13076	自我偏袒
arXiv:2307.03172	Lost in the Middle	arXiv:2410.21819	熟悉度偏袒
arXiv:2402.14848	Same Task More Tokens	arXiv:2310.01798	自纠失败
arXiv:2502.05167	NoLiMa	arXiv:2507.02778	自纠基准
arXiv:2505.06120	多轮迷失	arXiv:2306.05685	LLM-as-Judge

### 厂商实践与工程文献

Anthropic 《Building effective agents》· 《How we built our multi-agent research system》  
Anthropic 《Effective context engineering for AI agents》  
Cognition 《Don't Build Multi-Agents》· Chroma 《Context Rot》  
LangGraph Persistence · Temporal Durable Execution

引用策略：why 引论文，how 引厂商实践 + 本 repo 案例

# 谢谢

Q & A · 备用 slides B1–B8 候场

# 附录 • QA 弹药

---

备用 slides B1–B8（正文同质量标准）  
不计时，按提问取用

workspace/Andrej Karpathy \_ We\_re summoning ghosts\_ not building animals\_20260804\_232217/review/pyramid/summary.md

# Pyramid Gate Summary

Checkpoint	Report	Status	Score	Required revisions	Waiver reason
outline	outline.pyramid.json	pass	0.91	None	None
section_01	section_01.pyramid.json	pass	0.89	None	None
section_02	section_02.pyramid.json	pass	0.89	None	None
section_03	section_03.pyramid.json	pass	0.90	None	None
section_04	section_04.pyramid.json	pass	0.89	None	None
main	main.pyramid.json	pass	0.91	None	None

---

main.pyramid.json 结论区（集成后整篇 main.tex 校验）：

```
"status": "pass", "score": 0.912, "artifact_type": "tex_document", "context_label": "main"
```

同目录另有 outline.pyramid.json、section\_01-04.pyramid.json（共 6 份 checkpoint 报告，裁剪）

三关金字塔校验：outline → 四章 → main.tex 全 pass，结构质量事中拦截

.codex/hooks.json

```
{
 "hooks": {
 "Stop": [... delivery_guard.py hook-stop: 会话最后一厘米兜底（裁剪）...],
 "PreToolUse": [
 {
 "matcher": "Bash",
 "hooks": [
 {
 "type": "command",
 "command": "python .agents/skills/bilibili-render-pdf/scripts/latex_compile_pretooluse_guard.py",
 "commandWindows": "D:\\Project\\...\\latex_compile_pretooluse_guard.py" （同一守卫，绝对路径，裁剪）,
 "timeout": 5,
 "statusMessage": "Checking LaTeX compile guard"
 }
]
 }
]
 }
}
```

agent 想直接调 xelatex? PreToolUse hook 在 Bash 层拦下: 编译必须走受控 wrapper

待删除/long-running/9QebvrrY3KY\_acceptance\_criteria\_validate\_20260728\_184345\_fa37b8eb/status.json

```
{
 "child_pid": 51976,
 "elapsed_seconds": 1.062,
 "exit_code": 0,
 "finished_at": "2026-07-28T18:43:49.056+08:00",
 "heartbeat_at": "2026-07-28T18:43:49.060+08:00",
 "latest_output_at": "2026-07-28T18:43:48.871+08:00",
 "log_sizes": { "merged": 99, "stderr": 0, "stdout": 87 },
 "run_id": "fa37b8eb-a4e7-4763-a165-8fa442ab0f98",
 "schema_name": "persisted-command-status",
 "security": {
 "acceptance_evidence_eligible": true,
 "classification": "no_secret_detected"
 },
 "started_at": "2026-07-28T18:43:45.701+08:00",
 "state": "succeeded",
 "supervisor_identity": { "pid": 45076, "process_creation_identity": "windows-filetime:134297090258119624" },
 "supervisor_pid": 45076,
 "target_identity": { "pid": 51976, "process_creation_identity": "windows-filetime:134297090280815848" },
 "updated_at": "2026-07-28T18:43:49.060+08:00"
}
```

（单行 JSON 按原 key 顺序逐行展开，值未改动）

runner 流式扫日志：检出 secret → eligible=false，整份证据降级为本地诊断材料

跨 chunk 边界也不漏：保留上段尾部做扫描窗

```
777 class _StreamSecretDetector:
778 def __init__(self, secret_values: Sequence[str]) → None:
779 self._secret_values = secret_values
780 longest_secret = max(
781 (len(secret.encode("utf-8")) for secret in secret_values),
782 default=0,
783)
784 self._context_size = max(
785 SECRET_SCAN_CONTEXT_BYTES,
786 longest_secret - 1,
787)
788 self._context = b""
789
790 def observe(self, chunk: bytes) → bool:
791 scan_window = self._context + chunk
792 detected = _contains_detected_secret(scan_window, self._secret_values)
793 self._context = scan_window[-self._context_size :]
794 return detected
```

检出 secret → `acceptance_evidence_eligible = false`：整份证据降级为本地诊断材料，防证据链被污染

sessions/019f3f40-.../current.json

```
{
 "schema_version": "1.1",
 "scope": "session",
 "session_id": "019f3f40-b7be-7ff2-8161-19c80776a929",
 "stage": "ready_for_delivery",
 "video_output_dir": "workspace/Inside Anthropic_ ...",
 "target_file": "workspace/.../delivery_target.json",
 "source_skill": "youtube-render-pdf",
 "started_at": "2026-07-08T09:09:00+08:00",
 "updated_at": "2026-07-08T11:05:11+08:00"
}
```

(video\_output\_dir / target\_file 长路径裁剪)

原文: Inside Anthropic\_ How Claude Tag Is Changing Agentic Work\_20260708\_090900

.codex/delivery-targets/task-index.json - 开头 (裁剪)

```
{
 "schema_version": "1.0",
 "tasks": [
 {
 "video_output_dir": "待删除/.../field-guide-fable",
 "target_file": "待删除/.../delivery_target.json",
 "owner_session_id": "e2e-field-guide-fable-a",
 "owner_status": "blocked", "stage": "blocked",
 "updated_at": "2026-07-13T15:17:21+08:00"
 },
 ... 第 2 条: e2e-fable-5-usage-b / blocked (裁剪) ...
 {
 "video_output_dir": "workspace/..._20260708_090900",
 "owner_session_id": "019f3f40-b7be-7ff2-8161-19c80776a929",
 ... (后续条目裁剪)
 }
]
}
```

左：本会话交付锁；右：任务登记处——新 session 接管旧目录必须显式 handoff

adr/0020-...-task-envelopes.md

```
Use kernel-issued task envelopes for every subagent
```

The workflow requires several specialized subagents with different inputs, output paths, schemas, and write permissions. Repeating those details in prompts causes path drift, stale dependency use, freeform report schemas, and unnecessary token consumption. Acceptance already demonstrates the value of a generated skeleton and an explicit artifact boundary.

### ## Considered Options

- Keep role contracts entirely in skill prompts:  
**rejected** because prompts cannot prove current paths, fingerprints, or schema versions.
- Define a different handoff format for every role:  
**rejected** ... (理由裁剪)
- Give every subagent one kernel-issued task envelope with role-specific payloads: **selected** ... (裁剪)

(正文长行按窗口宽度换行, 措辞未改)

schemas/.../subagent-task-envelope.v3.schema.json

```
{
 "schema_name": { "const": "subagent-task-envelope" },
 "schema_version": { "const": "3.0.0" },
 "kernel_version": { "const": "2.0.0" },
 "role": { "const": "source_acquisition" },
 "task_stage": { "enum": ["provider_acquisition",
 "semantic_judgment",
 "whisper_transcription"] },
 "platform": { "enum": ["bilibili", "youtube"] },
 "required": [
 "schema_name", "schema_version",
 "kernel_version", "task_id",
 "logical_task_key", "role",
 "task_stage", "platform",
 "source_epoch", "authority_binding",
 "prepared_at", "task_root_path",
 "input_artifacts", "allowed_read_paths",
 "protected_run_snapshot", "write_set",
 "required_outputs", "generated_prompt",
 "bounded_semantic_fields",
 "whisper_audio_candidate",
 "resource_request", "fairness_group_id"
] }
```

kernel 颁发任务信封: 权责/读集/写集机器可校验——代码+测试齐备, 生产 cutover 未发生

.agents/skills/bilibili-render-pdf/SKILL.md – Long Video Strategy (L215–L224)

215   ## Long Video Strategy

217   For longer videos, do not rely on a single monolithic pass.

219   - If the video is longer than 20 minutes, or the subtitle file

contains more than 300 subtitle entries, split the work into smaller segments.

220   - Prefer chapter boundaries or 分P boundaries for splitting. ... (裁剪)

221   - When subagents are available, spawn multiple subagents in parallel ... (裁剪)

222   - Give each subagent a concrete segment boundary and require it to return:

the segment's teaching goal, the core claims, ... time provenance, ... (裁剪)

223   - Keep a small overlap between neighboring segments ... deduplicate during integration. (裁剪)

224   - The main agent must integrate the segment outputs into one unified outline ... (裁剪)

(L219 长行按窗口换行; L220–L224 行尾裁剪; 空行 L216/L218 未绘出)

视频 >20 分钟或字幕 >300 条 → 强制切段、并行 subagent、只回结构化摘要 (设计约定)



0001 use-codex-exec-for-pyramid-semant...	0023 version-all-workflow-contracts-an...	0045 separate-task-claim-reclaim-from...
0002 use-json-acceptance-contract-for...	0024 place-the-workflow-kernel-in-a-ro...	0046 serialize-canonical-promotion-wit...
0003 use-guarded-latex-compile-wrapper	0025 use-json-schema-as-the-kernel-con...	0047 treat-sqlite-sidecars-and-test-da...
0004 use-session-scoped-delivery-targe...	0026 materialize-fixed-reports-from-bo...	0048 prove-compile-input-closure-from...
0005 use-delivery-glossary-for-non-eng...	0027 separate-semantic-prompts-from-me...	0049 implement-the-kernel-in-thirteen...
0006 separate-delivery-ownership-from...	0028 split-final-acceptance-into-text...	0050 separate-target-design-acceptance...
0007 use-github-issues-for-human-super...	0029 make-cross-dimension-acceptance-f...	0051 activate-acceptance-v2-globally-w...
0008 use-one-video-workflow-kernel-wit...	0030 require-acceptance-report-v2-and...	0052 route-content-assurance-failures...
0009 use-per-video-workflow-run-record...	0031 plan-repairs-deterministically-an...	0053 seal-final-artifacts-and-render-e...
0010 use-phase-plus-checkpoint-graph-f...	0032 hide-the-content-production-dag-b...	0054 fail-closed-when-the-cross-run-co...
0011 isolate-source-acquisition-as-a-v...	0033 allow-one-incremental-figure-wave...	0055 coordinate-delivery-projections-t...
0012 use-two-phase-bootstrap-and-run-i...	0034 run-consistency-and-source-faithf...	0056 bind-final-acceptance-tasks-to-an...
0013 generate-artifact-instances-at-th...	0035 admit-tasks-through-fixed-resourc...	0057 adopt-multi-context-domain-docume...
0014 standardize-new-video-output-dire...	0036 make-batch-a-projection-over-inde...	0058 adopt-persisted-command-execution
0015 create-video-output-scaffolds-onl...	0037 schedule-resource-admission-with...	0059 run-project-tests-from-an-externa...
0016 enforce-an-end-to-end-windows-wor...	0038 start-implementation-with-an-offl...	0060 establish-target-only-delivery-qu...
0017 separate-run-identity-from-manual...	0039 test-workflow-behavior-at-three-p...	0061 materialize-precompile-quality-an...
0018 default-to-fresh-source-acquisiti...	0040 cut-over-to-the-kernel-one-platfo...	0062 reconcile-sealed-text-with-guarde...
0019 bound-source-agent-judgment-with...	0041 partition-acceptance-with-a-versi...	0063 materialize-target-only-acceptanc...
0020 use-kernel-issued-task-envelopes...	0042 use-sqlite-only-for-cross-run-tra...	0064 activate-global-acceptance-v2-gate

64 条 ADR: 给开发 harness 的 agent 用的 harness —— 高亮: 0005 术语表 / 0020 任务信封 · 0030 验收 v2 / 0058 落盘 · 0064 Global Gate

docs/handoffs/issue43-global-gate-cutover-20260804.md

# Issue #43 Global Gate Cutover handoff

## Resume boundary

Issue #43 remains in progress on branch  
`video-workflow-2.0` at committed HEAD  
`06c205e4921addab1af0863b723d4e9b5c0ffa54`.

Authoritative requirements remain GitHub  
Issue #43. The approved execution order is:

1. affected-module tests;  
2. fast static, contract, and `git diff --check`;  
3. independent spec / release / security reviews;  
4. code freeze;  
5. final validator and complete Delivery Evidence.

Any code repair after a review or validator  
failure returns to step 1.

## Current repair

The final evidence run exposed real Windows races  
in `AcceptanceV2CliTests.test\_two\_writers\_are\_ ...`

.../global-gate/exit-evidence-manifest.json

{  
 "kind": "video-workflow-exit-evidence",  
 "fingerprint\_algorithm": "sha256-raw-v1",  
 "slice": {  
 "number": 11,  
 "name": "global-acceptance-v2-gate"  
 },  
 "evidence\_paths": [ ... ],  
 ...  
}

2026-08-04 Global Gate 激活当日：一份 handoff 文档把 HEAD、执行顺序、当前修复、禁区完整移交给下一个 session

人类侧的长程任务同样靠「落盘 + 交接」活着 —— 与 agent 侧同构（真实文档节选，右为同日出炉的退出证据清单）

docs/adr/0064-activate-global-acceptance-v2-gate.md

# Activate the global Acceptance Report v2 gate

## Decision

Activate one repository-wide final-quality gate with status `active\_global\_gate`.

- Acceptance Report v2 is the sole machine-readable semantic delivery decision for Kernel and Legacy inputs.

- A Kernel input retains its real Run Record, Control Store authority, ...

- A Legacy directory enters through a fresh, explicitly named, Run-record-free Legacy Acceptance Input Set. It uses the same materializer, report contract, decision rules, and active Delivery Guard.

（真实 ADR 节选，省略处以 ... 表示）

evidence/global-gate/exit-evidence-manifest.json

{

"kind": "video-workflow-exit-evidence",

"schema\_version": 2,

"fingerprint\_algorithm": "sha256-raw-v1",

"slice": {

"number": 11,

"name": "global-acceptance-v2-gate"

},

"evidence\_paths": [

"evidence/global-gate/logs/...tests.log",

"evidence/global-gate/persisted/.../status.json",

"evidence/global-gate/persisted/.../exit-code.txt",

...（5 组日志 + 5 组 persisted 三件套）

]

}

权威分裂 = 自评偏差的制度形态。对策：**单一权威**——v2 是唯一机器可读交付决策，禁止双轨、禁止 v1→v2 翻译  
26 份 v1 报告判为历史；Legacy 目录经 adopt 进同一道门（ADR 0030/0051/0064；Slice 11 退出证据已落盘）